

Highlights

R4Tun: LLM-guided adaptive segmental tunnel lining segmentation in point clouds

Xinghui Tao, Guangming Wang, Jelena Ninić, Brian Sheil

- Introduce R4Tun that tunes tunnel lining segmentation when conditions shift.
- Stage agents adapt expert pipeline parameters using memory (m), state (s), and knowledge (k).
- Across 5 tunnel subsets, $m + s + k$ raised overall mean mIoU from 0.18 to 0.43–0.48.
- Ablations suggest state provides the main increment; knowledge adds a smaller complex-tunnel gain.
- R4Tun supports controlled, label-free, cross-LLM adaptation of an expert pipeline in the tested setting.

R4Tun: LLM-guided adaptive segmental tunnel lining segmentation in point clouds

Xinghui Tao^a, Guangming Wang^{a,*}, Jelena Ninić^b and Brian Sheil^a

^a Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, United Kingdom

^b Department of Engineering, Durham University, Stockton Road, Durham, DH1 3LE, United Kingdom

ARTICLE INFO

Keywords:

Segmental tunnel lining
Point cloud segmentation
Tunnel inspection
Large language models
Parameter adaptation
Multi-agent systems

ABSTRACT

Automated inspection of segmental tunnel linings requires adaptive segmentation from 3D point clouds, yet expert-tuned pipelines often degrade when tunnel conditions vary. This paper presents R4Tun, a large language model (LLM)-driven adaptation framework that extends an expert-designed pipeline (SAM4Tun) with bounded parameter tuning informed by structured context: memory (m), state (s), and knowledge (k). Evaluated on 5 selected Seg2Tunnel subsets (3 regular, 2 complex) across three LLMs, the full $m + s + k$ design raised mean Intersection-over-Union (mIoU) from 0.18 to 0.43–0.48 and overall accuracy (OA) from 0.42 to 0.59–0.65 relative to the static SAM4Tun baseline. On complex tunnels, mIoU rose from 0.04 to 0.15–0.20. Across 45 (5 tunnels $\times$ 3 different LLMs $\times$ 3 context settings) runs, the LLMs showed similar parameter-adjustment trends (with overlapping 95% CIs on mean gains) and consistently adjusted a shared set of critical parameters. These results support R4Tun as a controlled, label-free, cross-LLM adaptation mechanism in the tested SAM4Tun–Seg2Tunnel setting, demonstrating consistent accuracy gains.

1. Introduction

Automated inspection of segmental tunnel linings is required to support structural health assessment and long-term operational safety, given the scale, frequency, and access constraints of modern tunnel networks [1]. In practice, engineers routinely encounter projects in which tunnel geometry, lining properties, and data acquisition settings vary [2]. Furthermore, although modern laser scanning enables high-fidelity tunnel capture, the resulting measurements often contain mixed structural elements, occlusions, and noise, making direct extraction of lining components challenging [3, 4]. Consequently, segmentation methods need to be adaptive enough to handle this complexity and variability [5, 6].

Existing tunnel point-cloud segmentation methods are grouped into three broad categories: feature engineering, deep learning and foundation-model pipelines (see Fig. 1 and Section 2). Feature engineering rules are interpretable but lack robustness under changed conditions [2]. Supervised deep-learning models generalise through data but require large labelled datasets and periodic retraining, while lacking auditability [7, 8, 9]. Foundation-model pipelines bypass annotation by using models pre-trained on large datasets [10, 11], but remain sensitive to expert-tuned processing parameters. Existing tunnel-segmentation approaches have not yet demonstrated, in a single workflow, both robust adaptation to changed conditions and an auditable adaptation process. Among foundation-model

pipelines, SAM4Tun [12] combines point-cloud preprocessing with SAM-based prompting [11] and has shown strong performance under expert tuning. However, the pipeline is highly sensitive to its many processing parameters. When tunnel geometry or scanning conditions depart from the tuning reference, performance can degrade, and identifying which parameters need adjustment can require repeated expert intervention.

Given that recent LLMs can follow multi-step instructions over structured inputs, parameter adaptation can be formulated as a diagnostic task in which the model receives context and proposes bounded adjustments. LLMs have been applied to engineering workflows including chemical synthesis planning [?], multi-agent coordination for complex project tasks [13, 14, 15], and manufacturing decision support [16]. However, these studies do not address parameter adaptation in infrastructure inspection pipelines. In this study, we propose R4Tun, an LLM-guided adaptation system that extends the SAM4Tun pipeline by adjusting stage parameters in response to changing tunnel conditions. The framework provides each agent with structured context $m + s + k$ (memory, state, and knowledge), from which it produces bounded parameter updates via stepwise prompting. We evaluate whether this contextual information improves segmentation adaptability across both regular and complex tunnels, and whether the resulting adaptation behaviour is consistent across LLMs. To isolate the contribution of the LLM beyond deterministic tuning, we also benchmark R4Tun against a rule-based adaptation derived from the same per-stage knowledge documents.

We position R4Tun as a mechanism contribution rather than as a deployable final-inspection system.

*Corresponding author

✉ gw462@cam.ac.uk (G. Wang)

ORCID(s):

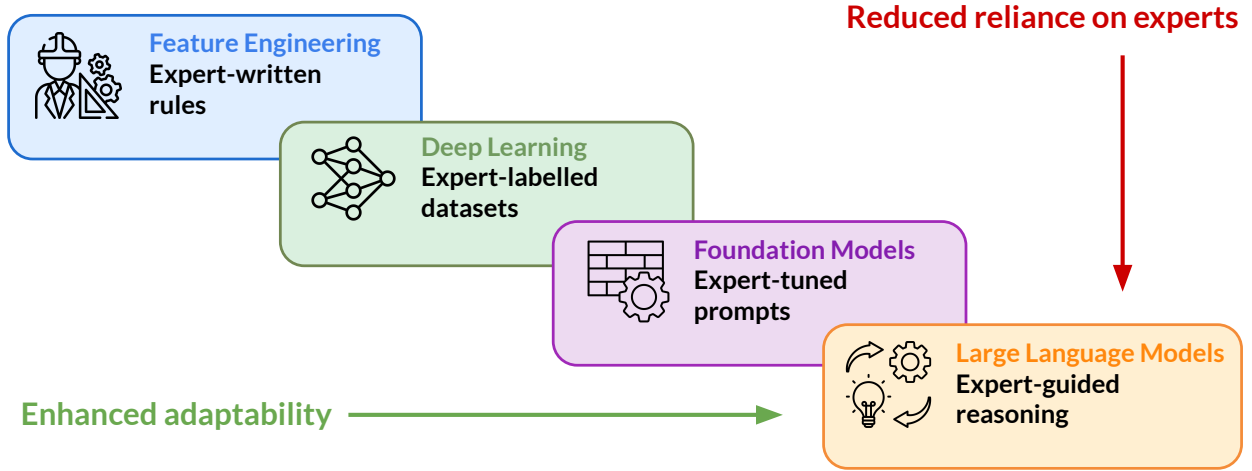


Figure 1: Segmentation approaches arranged by their adaptation mechanisms. Expert-guided, LLM-driven methods can extend foundation-model pipelines by adding parameter adaptation with structured context and encoded expert knowledge. In this work, we test one such mechanism (R4Tun) under controlled, label-free, cross-LLM settings while keeping the underlying SAM4Tun pipeline fixed.

The strength of the framework lies in integrating LLM-driven bounded parameter adaptation with an expert-designed pipeline. We study whether structured context helps an LLM re-parameterise the pipeline under controlled, label-free, cross-LLM conditions; the process is auditable through logged rationales and bounded parameter updates. Absolute segmentation accuracy remains bounded by the open-source SAM4Tun ceiling and by the single expert reference used in this experiment. These constraints are discussed in the Error Analysis subsection (Section 4.6).

This paper makes the following contributions:

1. A framework design in which LLM agents adapt the parameters of the SAM4Tun pipeline using structured context $m + s + k$: memory of the reference configuration, state from intermediate pipeline outputs, and a knowledge document of tunnel category-specific configuration shared across all tunnels.
2. Empirical evidence from cumulative ablation across 5 tunnels suggesting that intermediate pipeline state is an important contributor to adaptation, while the knowledge component adds a smaller increment concentrated on the complex category.
3. Cross-LLM evidence showing that three different LLMs (Opus-4.6, GPT-5.4, Gemini-3-Flash) produce overlapping effect ranges and adjust a consistent subset of critical parameters under the same prompt structure.
4. Evidence that deterministic, rule-based adaptation explains part, but not all, of the observed mIoU increase, which is consistent with an additional contribution from the LLM-based adaptation process in the tested setting.

The rest of this paper is organised as follows. Section 2 reviews related work. Section 3 presents methodology, including the R4Tun architecture, dataset, experimental design, and evaluation. Section 4 reports results. Section 5 discusses findings, implications, and limitations. Section 6 concludes.

2. Related work

This study situates R4Tun within tunnel point-cloud segmentation by contrasting (i) feature-engineered methods and supervised deep learning, (ii) foundation-model pipelines that reduce annotation but remain parameter-sensitive, and (iii) LLM reasoning as a way to produce auditable, bounded per-tunnel parameter updates.

2.1. Feature engineering versus deep learning

Tunnel point-cloud segmentation has advanced through two generations of methods, which trade off between audibility and adaptability. Feature-engineering approaches encode domain knowledge into deterministic geometric rules: centreline fitting via RANSAC [17], density-based surface clustering [18], and point-sampled surface processing (e.g., simplification and resampling) [19]. Those methods remain widely used in safety-critical inspection because their logic is explicit and auditable [2]. However, each rule embeds assumptions about tunnel geometry and point-cloud characteristics (diameter, ring spacing, joint pattern, point density), and changing any of these requires manual reconfiguration by a domain expert. Supervised deep-learning methods, such as PointNet++ [20], RandLA-Net [21], Mask3D [22], and TD3D [23], learn features directly from annotated point

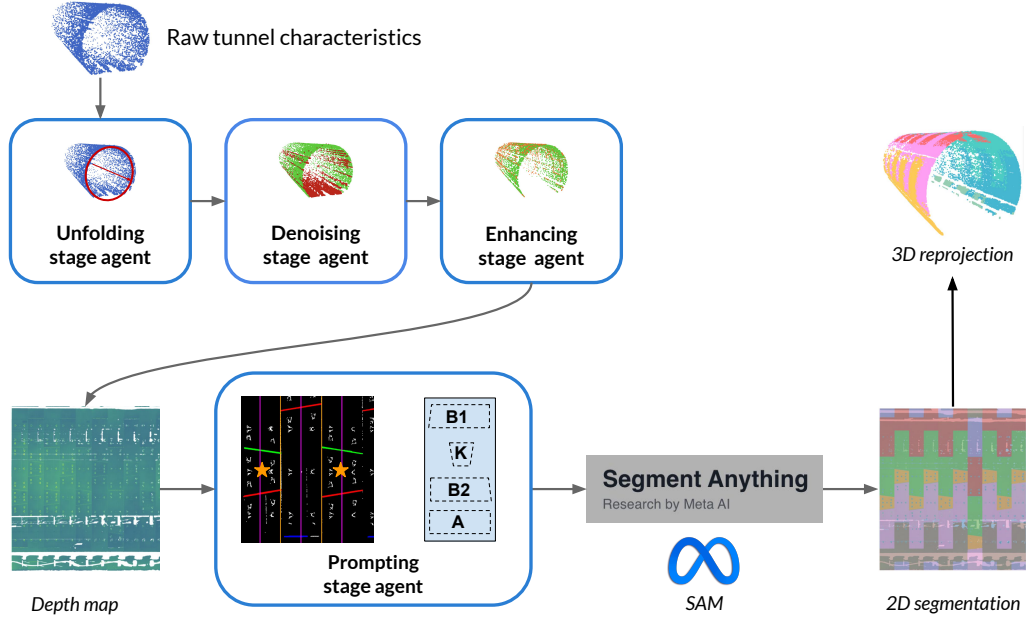


Figure 2: The R4Tun multi-agent architecture for tunnel segmentation. Four stage agents (unfolding, denoising, enhancing, and segmenting) adapt their parameters through LLM reasoning to generate segmentation.

clouds and can generalise across similar conditions present in the training set. Recent work has also extended tunnel and infrastructure segmentation through attention-based point-cloud models [?] and, for 2D inspection, YOLO-based image defect segmentation [?] [?]. Yet supervised deep-learning models require large labelled tunnel datasets that remain scarce [9], demand retraining for new tunnel conditions, and provide no mechanism for an engineer to trace or override a segmentation decision [8].

Therefore, for inspection operators, deploying to a new tunnel type requires either re-engineering rules, collecting new training data, or accepting degraded performance without a systematic diagnostic mechanism.

2.2. Foundation-model pipelines

To the best of our knowledge, direct 3D point-cloud segmentation of infrastructure at tunnel scene scale remains an open challenge for current foundation models. Existing 3D foundation models target common object recognition on curated datasets and do not transfer to large, noisy tunnel point clouds [24, 25, 26, 27]. As a result, practical tunnel pipelines often project 3D data into 2D representations to use mature vision foundation models pre-trained on large-scale image datasets, thereby bypassing the annotation bottleneck [28?]. SAM [11] performs class-agnostic segmentation from spatial prompts without task-specific training, and extensions such as SAM2 [29] for temporal consistency and SEEM [30] for natural-language-guided segmentation are broadening prompt-based segmentation further. These models have been adopted in infrastructure

contexts including crack detection [?] and Scan-to-BIM workflows [31?].

For tunnel linings, SAM4Tun [12] combines geometric preprocessing (unfolding, denoising, enhancing) with foundation-model 2D segmentation, achieving strong segmentation without training data. However, this pipeline depends on numerous stage-specific parameters that encode assumptions about the reference tunnel’s diameter, ring length, point density, and joint patterns. When a new tunnel departs from the reference, these parameters become misspecified and the pipeline can degrade without indicating which parameters fail or why. This sensitivity is not unique to SAM4Tun: most multi-stage pipelines that chain domain-specific preprocessing with a foundation model tend to inherit it. While foundation models have reduced reliance on labelled data, they have made the pipeline’s dependence on hand-tuned parameters more explicit, without removing the need for expert parameter tuning.

2.3. LLM reasoning

Recent LLM advances have introduced explicit reasoning capabilities relevant to the parameter-adaptation problem. Reasoning-oriented training [32, 33, 34, 35, 36, 37] produces models capable of decomposing complex problems and performing prompted consistency checks. Chain-of-thought (CoT) reasoning [38, 39] enables step-by-step logical traces. Multi-agent architectures coordinate specialised roles through shared context [14, 13, 15, 40, 41], and context engineering influences reasoning quality through the careful design of information provided to models [42, 43, 44].

In tunnelling segmentation, however, no prior work has applied LLM-based reasoning to a challenge in expert-designed pipelines: re-tuning a parameter-sensitive system to new conditions while preserving the engineer's ability to inspect and override each decision. We therefore introduce an LLM-based reasoning framework that retunes parameter-sensitive pipelines per tunnel using expert-guided context and a logged rationale for each proposed change. Meanwhile, whether LLM reasoning produces gains beyond what hand-coded rules can deliver from the same knowledge text remains an open question. Our experimental design therefore tests this directly by including a deterministic rule-based adaptation of those documents as a non-LLM control.

3. Methodology

Our methodology defines the tunnel-lining segmentation task, summarises the fixed SAM4Tun pipeline, and specifies the R4Tun adaptation layer (structured context $m + s + k$ and bounded parameter updates), followed by the dataset, baselines, ablations, metrics, and sensitivity analysis.

3.1. Task definition

The task is semantic segmentation of segmental tunnel linings from terrestrial laser scanning (TLS) point clouds. Given a raw point cloud of a tunnel section, the goal is to assign each point a structural label: background, key segment (K), base segments (B1, B2), and adjacent segments (A1–A3), with an additional A4 class for 7-segment complex tunnels. The unit of analysis is a tunnel subset spanning multiple rings selected from the Seg2Tunnel benchmark.

A key limitation is that the fixed configuration of the open-source SAM4Tun achieves a mean mIoU of 0.27 on regular tunnels, but only 0.04 on complex tunnels whose geometry departs from the tuning reference. As noted above, the pipeline provides no built-in signal indicating which parameters become misspecified.

3.2. R4Tun

R4Tun supplements the SAM4Tun pipeline [12] with an LLM-guided adaptation layer that adjusts parameters per tunnel without modifying the underlying algorithms (Fig. 2). Given a new tunnel point cloud, a characteriser first extracts raw geometric properties (diameter, density, coordinate ranges; full field list in Appendix B). Each of the four stages (unfolding, denoising, enhancing, and segmenting) has a dedicated LLM agent that adapts its parameters. In the segmenting stage, the adapted prompts are passed to SAM, and the resulting 2D masks are reprojected onto the 3D point cloud without further LLM intervention.

Stage 1-Unfolding (Fig. 3) establishes a cylindrical coordinate system for the tunnel. It slices the point cloud at regular longitudinal intervals, fits an ellipse to

each cross-section via RANSAC [17], and selects the model with the highest inlier support. The per-slice ellipse centres are interpolated into a smooth centreline that defines the cylindrical frame (r, θ, h) . The 3D tunnel surface is then projected into a 2D panoramic image in which each pixel encodes the radial distance to the centreline, producing a stable depth representation for downstream filtering and segmentation.

Stage 2-Denoising (Fig. 4) removes non-structural artefacts (rails, cables, and scattered points) that would otherwise interfere with segmentation. The algorithm applies grid-based radial-density filtering inspired by the DBSCAN clustering principle [18], grouping points by local density in cylindrical coordinates. Dense, connected regions are retained as tunnel surfaces while isolated points are rejected as noise. A pair of radial masks $(R_{\text{low}}, R_{\text{high}})$ constrains filtering to the expected tunnel radius, preserving joint boundaries and ring edges while discarding outliers beyond the lining surface. The result is a clean, continuous surface representation suitable for curvature analysis and interpolation in the next stage.

Stage 3-Enhancing (Fig. 5) improves geometric continuity and surface completeness before projection into the 2D depth map. Local curvature is estimated on neighbourhood points to guide point insertion: new points are placed between neighbours whose curvature difference is small (smooth panel regions), while high-curvature areas (joint boundaries and ring edges) are preserved intact. A three-stage progressive upsampling scheme inserts additional points at progressively finer resolutions to maintain surface continuity without blurring structural boundaries. Pixel-level interpolation then refines outlier points at joint locations. The enhanced surface is projected into a panoramic depth map that balances smooth panel regions with sharply defined joint boundaries.

Stage 4-Segmenting (Fig. 6) combines boundary detection and SAM segmentation into a single workflow. First, a Hough-transform-based detector [45] extracts linear features from the enhanced depth map that correspond to ring joint boundaries. Detected lines are filtered by orientation (oblique, horizontal, vertical) with separate Hough thresholds and merged when closer than a distance tolerance. The confirmed boundaries are then used to construct template-based prompts for key, base, and adjacent segments, which are passed to SAM [11]. SAM produces 2D segment masks on the panoramic image, which are reprojected into 3D point labels via the stored pixel-to-point mapping from Stage 1.

The pipeline contains 81 tunable processing parameters, spanning geometric defaults, filtering and interpolation settings, and boundary-detection thresholds. All parameters were expert-tuned on a reference tunnel (diameter 5.60 m, density 2,466 pts/m³, mIoU = 0.531); full parameter tables are provided in Appendix A. In

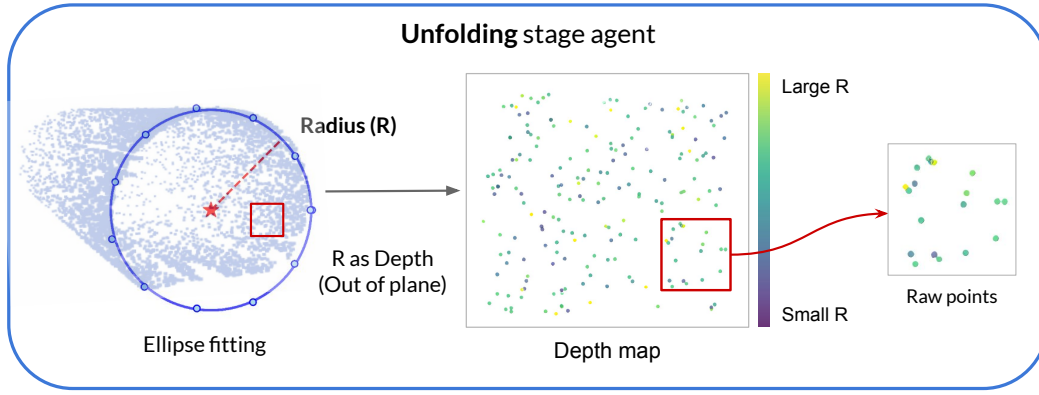


Figure 3: Stage 1-Unfolding. Left: fitted tunnel cross-section used to determine the reference centre and radius for cylindrical unfolding. Centre: 2D panoramic depth map derived from the 3D point cloud. Right: zoomed region of the unfolded map showing radial-distance encoding.

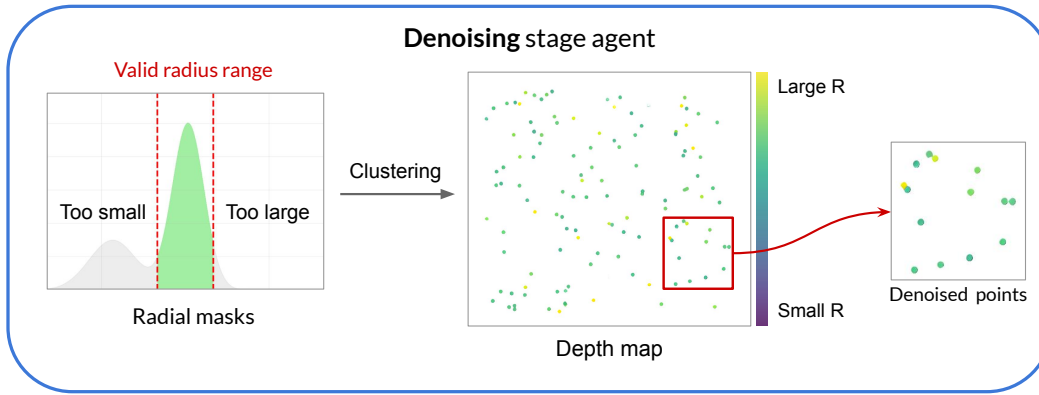


Figure 4: Stage 2-Denoising. Left: radial-distance distribution used to filter points outside the expected tunnel lining surface. Centre: unfolded 2D depth map after filtering, showing the cleaned point distribution. Right: zoomed region of the denoised map.

this study, the underlying pipeline stages remain fixed across all conditions; only the parameter values change. The SAM4Tun baseline applies this single expert-tuned configuration uniformly to all 5 tunnels without per-tunnel adaptation, serving as the static baseline against which LLM-guided adaptation is measured.

3.3. Agent design

Each of the four stage agents is a self-contained unit comprising (i) a *context* (Section 3.3.1; Fig. 7a) and (ii) a Python *analyst* that constructs the full LLM prompt and parses the returned JSON. For each agent-driven stage, the analyst assembles a prompt from the current context level, calls one of the selected LLM APIs (Opus-4.6, GPT-5.4, or Gemini-3-Flash) to follow a five-step CoT protocol (Section 3.3.2; Fig. 7b): referencing, diagnostic inspection, parameter adaptation, validation, and JSON output. Each agent returns a single schema-conformant parameter JSON, which the corresponding pipeline stage executes unchanged. A characteriser plugin then extracts statistics from the stage output, updating the cumulative state available to subsequent stages. Each stage's parameters are adapted from the expert-tuned reference configuration

together with this cumulative state; the stage scripts and evaluation code are identical across all ablation conditions, with only the parameter JSONs changing. Because every parameter change is logged alongside a generated textual rationale, engineers can review and, where necessary, override adaptation decisions. A worked example is provided in Appendix D. After the final segmentation, per-point labels are evaluated against ground truth using mean Intersection over Union (mIoU) as the primary metric.

3.3.1. Context design

Each stage agent receives structured context comprising three components: memory, state and knowledge (Fig. 7a). A worked example of the three components for the denoising agent is given in Appendix C.

Memory (Appendix C.1) stores the reference tunnel's characteristics (a JSON file containing geometry, point density, coordinate ranges, nearest-neighbour distances) alongside the expert-tuned reference parameters. The agent compares the current tunnel's characteristics against this reference to quantify deviation and adjust its parameters via CoT reasoning (Section 3.3.2). Memory is static: it does not change between stages.

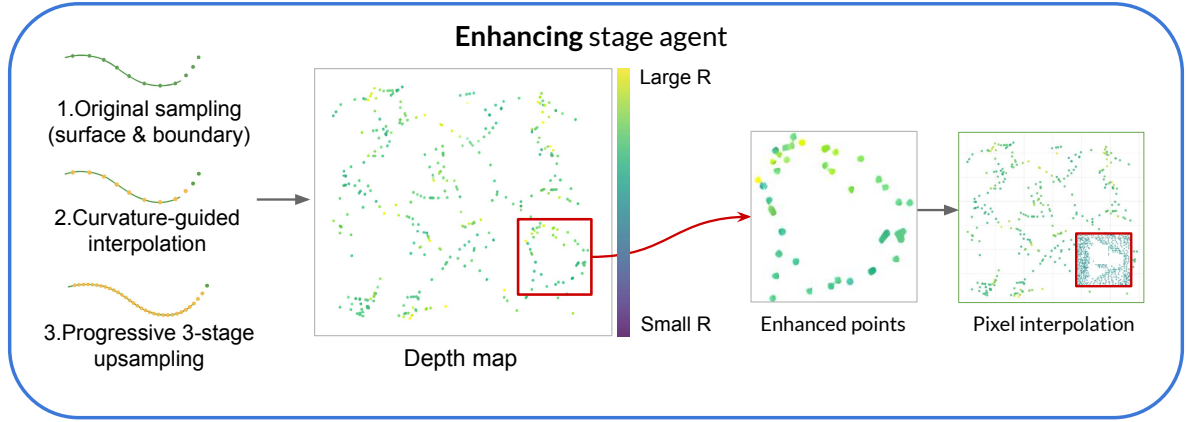


Figure 5: Stage 3-Enhancing. Left: curvature-guided surface and boundary interpolation along the tunnel lining, showing original sampling, curvature-guided insertion, and progressive three-stage upsampling (shown as three sequential upsampling steps). Centre: unfolded 2D depth map after enhancement, where inserted points improve geometric continuity; the red box highlights a local region of interest. Right: zoomed views of the highlighted region, showing added points and pixel-level interpolation.

State (Appendix C.2) captures cumulative geometric and statistical properties after each pipeline stage executes. These JSON summaries are injected into subsequent agents' prompts, and the state grows as stages execute. Importantly, state alone does not prescribe parameter changes: the numeric summaries of the actual point distribution after upstream processing must be interpreted in the context of parameter semantics and tunnel conditions before they can inform parameter updates.

Knowledge (Appendix C.3) supplies domain-specific guidance in human-readable markdown documents. Each stage has its own knowledge document covering three types of guidance: (i) cross-tunnel variation, which describes common lining layouts and structural conditions; (ii) parameter notes, which define each tunable parameter, its empirically validated range, and proven defaults; and (iii) diagnostic rules, which provide constrained guidelines for recommended parameter adjustments. Knowledge is authored once and shared across all tunnels.

3.3.2. CoT design

CoT reasoning provides the analytical structure for each agent's decision-making (Fig. 7b). The agent is prompted to follow a five-step protocol (Algorithm 1). A worked example of the full five-step trace is provided in Appendix D.

1. *Referencing*: The agent compares the current tunnel's characteristics against the stored reference to quantify deviation (e.g., +36% diameter increase).
2. *Diagnostic inspection*: The agent attributes the deviation to a plausible cause and identifies which specific tunable parameters are implicated. If

signals conflict, the agent is instructed to prioritise *State* over *Memory*, as State summarises geometry after upstream processing.

3. *Parameter adaptation*: The agent proposes updates for the implicated parameters. These updates are instructed to stay within the empirical ranges defined in the *Knowledge* component.
4. *Validation*: A self-correction step performs a consistency check to confirm that the proposed values satisfy logical constraints (e.g. a radius lower bound must be smaller than the corresponding upper bound). If a constraint is violated, the value is prompted to move toward to the nearest valid bound. This check-and-correction is executed as an LLM reasoning step, rather than a deterministic software routine, and therefore does not guarantee constraint satisfaction.
5. *JSON output*: The selected parameters and their reasoning trace are packaged into a single schema-conformant JSON object.

Each stage agent reads the tunnel characteristics and the latest pipeline state to identify deviations and decide which parameters to adjust. Starting from the expert reference value, it computes an updated setting from the post-stage statistics and clips it to the empirically observed range in the knowledge document when needed. This differs from the deterministic rule baseline (Appendix E), which assigns each tunnel to a discrete category based on raw characteristics and then applies one fixed parameter value per family. By contrast, the LLM diagnoses parameters on the fly, resolves conflicts between live state and static memory, and adapts values continuously from the observed statistics instead of emitting constants.

Notation for Algorithm 1. C_{target} denotes the raw characteristics of the current tunnel, while M stores the reference configuration ($C_{\text{ref}}, P_{\text{ref}}$). S is the

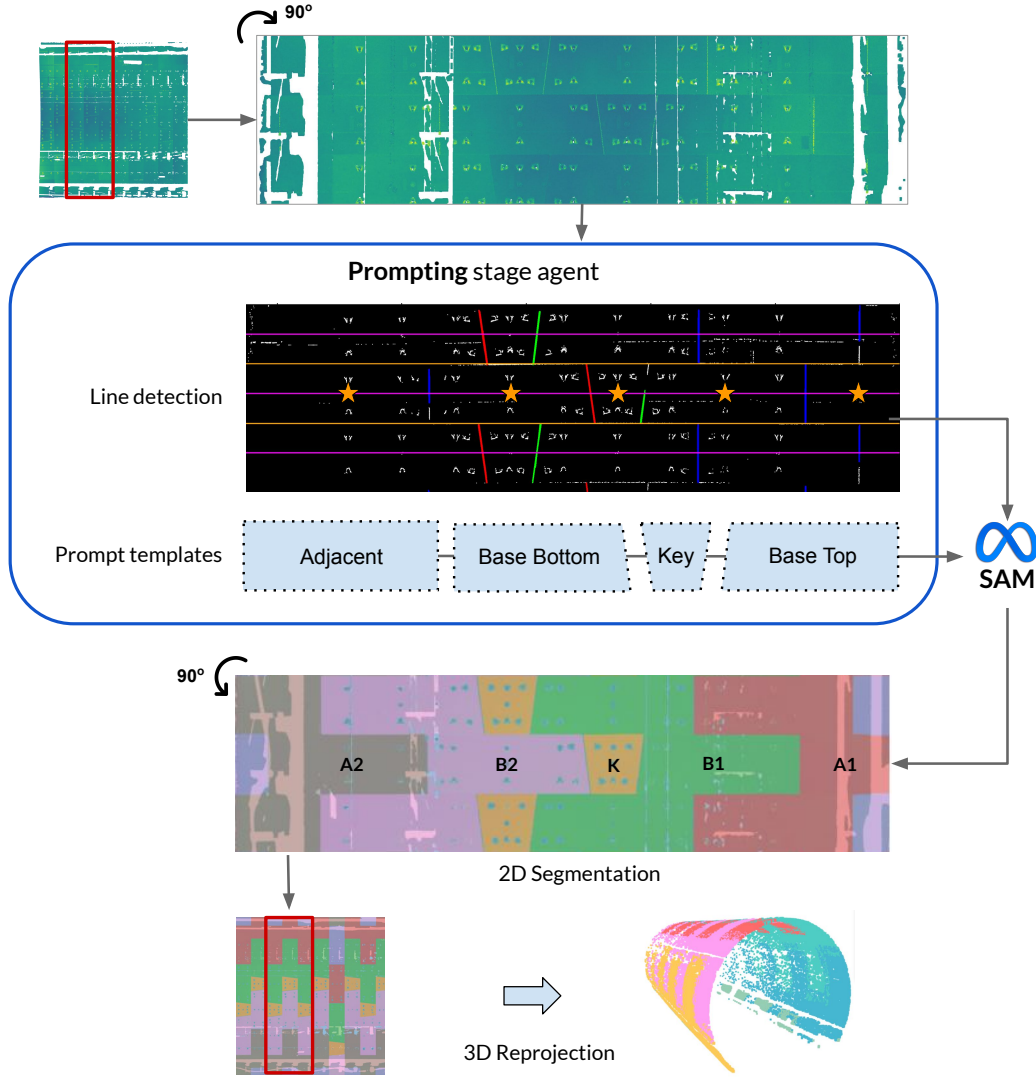


Figure 6: Stage 4—Segmenting. The segmenting stage applies Hough-transform line detection [45] to identify ring boundaries, then constructs template-based prompts and feeds them to SAM [11]. SAM produces 2D segment masks that are reprojected into 3D. **K** denotes the key segment, **B** denote base segments, and **A** denote adjacent segments around the ring. (Note: rotation is shown for visualisation purposes only.)

cumulative pipeline state, and S_{current} is its most recent summary. K is a shared knowledge document that encodes parameter semantics, per-parameter admissible bounds B , and cross-parameter constraints R . The adaptation procedure computes a geometric deviation descriptor Δ_{geom} , selects an implicated parameter subset $P_{\text{implicated}}$, proposes an updated parameter map P_{proposed} (indexed as $P_{\text{proposed}}[p]$), and finally returns the schema-formatted JSON object P_{adapted} .

3.4. Experimental setup

3.4.1. Dataset

We selected 5 tunnel subsets from the Seg2Tunnel benchmark (Table 1). These subsets cover the main variations in segmental tunnel geometry (diameter, ring

length, segment count, key-segment layout, and point-density distribution), providing a controlled basis for evaluating adaptation across tunnel categories.

As shown in Fig. 8, staggered and continuous tunnels are grouped as “regular” ($n = 3$): they share the 5.60 m nominal diameter, 1.2 m ring spacing, six segments per ring, and, most importantly, regular patterns of key-segment positions similar to the reference. Complex tunnels ($n = 2$) depart from that reference in multiple coupled ways: nominal inner diameter increases by 34% (5.60 m $\rightarrow$ 7.50 m), ring length by 50% (1.2 m $\rightarrow$ 1.8 m), and each ring adds a seventh segment with an interleaved key-segment arrangement that does not repeat ring-to-ring. In addition, off-axis scanning yields non-uniform sampling, partial circumferential coverage, and angle-dependent range and incidence. Together, these differences alter both the geometry

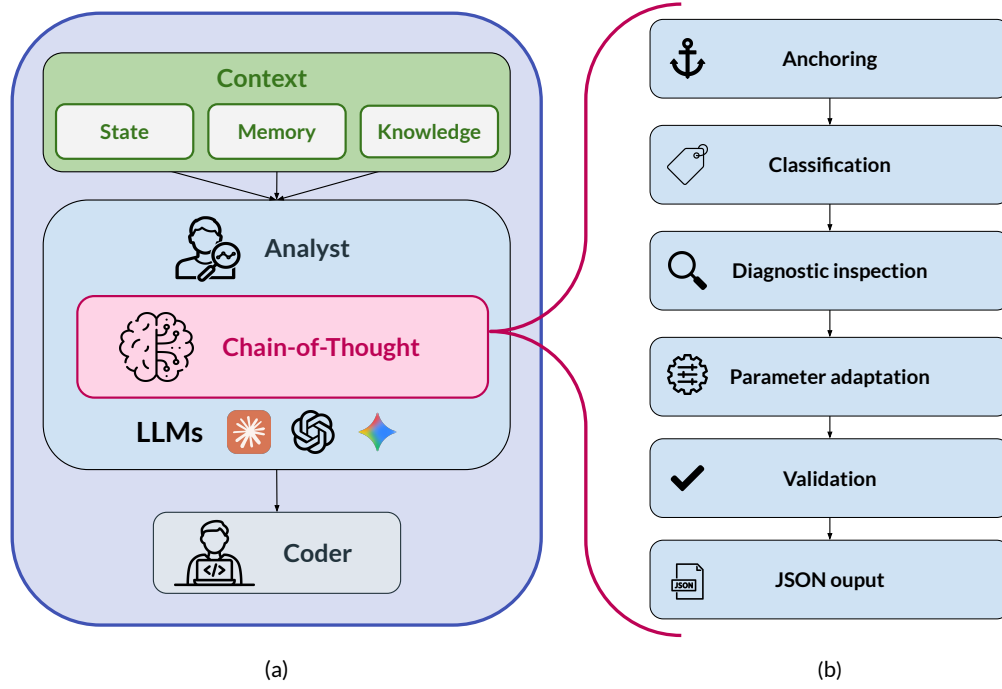


Figure 7: The left column (a) shows R4Tun stage agent architecture. Each agent maintains a shared and cumulative context that informs an LLM analyst component performing CoT reasoning. The right column (b) shows the five CoT phases: (1) Referencing, (2) Diagnostic inspection, (3) Parameter adaptation, (4) Validation, and (5) JSON output.

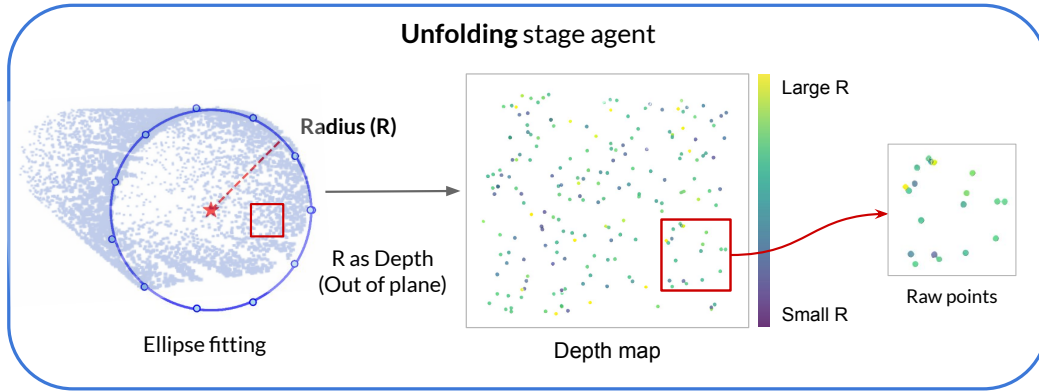


Figure 8: Tunnel lining pattern categories. From left to right: regular (staggered) tunnels exhibit a repeating ring-to-ring pattern; regular (continuous) tunnels maintain a fixed segment order around each ring; complex tunnels show no consistent repeating pattern.

visible in the point cloud and the semantic targets the pipeline must recover. Consequently, a fixed parameter set cannot be assumed to transfer from the regular reference; these conditions constitute the primary stress test for whether R4Tun can adapt the same algorithmic pipeline to off-design tunnel conditions. Ground-truth labels are per-point structural class annotations provided by the Seg2Tunnel benchmark; they are used for evaluation only and are not provided to the LLM or pipeline during adaptation.

3.4.2. Experimental design

The experiment follows a cumulative ablation design with four conditions, each adding one context component (Table 2). This design isolates the cumulative contribution of each component while maintaining a controlled comparison. Level 0 (SAM4Tun) is the fixed expert-tuned baseline with no LLM involvement. Levels 1–3 progressively provide the LLM with richer context, enabling the cumulative ablation to attribute improvements to specific components. A separate *Non-LLM* condition (level 0a in Table 2) is included as the Non-LLM adaptive control, allowing the LLMs' contribution to be read directly from the gap

Table 1
Dataset properties.

Property	Regular (T1, T2, T3)		Complex (T4, T5)
	Staggered (T1, T2)	Continuous (T3)	
Inner diameter	5.60 m	5.60 m	7.5 m
Ring length	1.2 m	1.2 m	1.8 m
Segments/ring	6	6	7
Joint type	Staggered	Continuous	Complex interleaved
Key-segment position	A ring-to-ring repeating pattern	A fixed ring-wise order	No repeating pattern
Scanning	Single-station	Multi-station	Single-station, offset
Eval. schema	6-class	6-class	7-class
Count	2 subsets	1 subset	2 subsets

Algorithm 1 LLM-driven Parameter Adaptation via CoT

Require: Target tunnel raw characteristics C_{target}
Require: Memory M (reference characteristics C_{ref} , reference parameters P_{ref})
Require: State S (cumulative pipeline outputs / intermediate summaries)
Require: Knowledge K (parameter semantics, tunable bounds B , and cross-parameter constraints R)
Ensure: Adapted stage parameter JSON P_{adapted}

- 1: **Phase 1: Referencing**
- 2: $\Delta_{\text{geom}} \leftarrow \text{Compare}(C_{\text{target}}, C_{\text{ref}})$ $\triangleright$ Quantify deviation from reference
- 3: **Phase 2: Diagnostic inspection**
- 4: $S_{\text{current}} \leftarrow \text{ExtractLatest}(S)$ $\triangleright$ Most recent stage statistics
- 5: *Reconciliation:* If S_{current} conflicts with M , prioritise S_{current}
- 6: $P_{\text{implied}} \leftarrow \text{IdentifyParameters}(\Delta_{\text{geom}}, S_{\text{current}}, K)$
 $\triangleright$ Select parameters relevant to observed deviations
- 7: **Phase 3: Parameter adaptation**
- 8: $P_{\text{proposed}} \leftarrow \emptyset$
- 9: **for** each parameter $p \in P_{\text{implied}}$ **do**
- 10: $P_{\text{proposed}}[p] \leftarrow \text{Adjust}(p, \Delta_{\text{geom}}, S_{\text{current}}, K)$ $\triangleright$
Propose new value within B guided by K
- 11: **end for**
- 12: **Phase 4: Validation**
- 13: **for** each parameter $p \in P_{\text{proposed}}$ **do**
- 14: $P_{\text{proposed}}[p] \leftarrow \text{SelfValidate}(P_{\text{proposed}}[p], p, K)$
 $\triangleright$ Execute an LLM internal reasoning step conditioned on K (semantics, bounds spec B , constraints R).
- 15: **end for**
- 16: **Phase 5: JSON output**
- 17: $P_{\text{adapted}} \leftarrow \text{FormatAsJSON}(P_{\text{proposed}})$
- 18: **return** P_{adapted}

between the *Non-LLM* column and the LLM columns of Table 4. We codified the per-stage knowledge documents into a deterministic Python rule table as a non-LLM baseline. It maps characterisation fields (e.g.,

Table 2
Ablation conditions.

Level	Code	Condition	What the LLM sees
0	SAM4Tun	Baseline	Default parameters
0a	Non-LLM	Rule table	18 parameters set by lookup
1	m	Memory	Reference characteristics + parameters
2	m+s	Memory+State	+ intermediate pipeline outputs
3	m+s+k	Memory + State + Knowledge	+ domain knowledge

diameter, ring length, segments-per-ring, joint type, point density, and station configuration) to the 18 critical parameters adjusted by the LLMs (Table 8) using explicit “if... then...” rules. The table and script are fixed for all 5 tunnels (regular and complex), with no evaluation-set tuning or subset-specific settings, isolating the value of LLM per-tunnel reasoning. The complete rule specification is provided in Appendix E.

To assess whether adaptation behaviour depends on a specific LLM, each condition was run with three different LLMs (Opus-4.6, GPT-5.4, and Gemini-3-Flash) under identical prompts. Across models, the pipeline code, evaluation scripts, and prompt structure were held constant, while the context content varied by ablation level. Other than setting temperature to 0 to minimise stochasticity (Table 3), all other API settings used vendor defaults. **This corresponds to 45 unique LLM-adaptation configurations: 5 tunnels $\times$ 3 LLMs $\times$ 3 context settings (m, m+s, m+s+k). Each configuration was executed a second time for repeatability checking, yielding $45 \times 2 = 90$ pipeline executions, plus 5 baseline runs with fixed parameters. Headline analyses (e.g., Table 4) use the first execution only.** All three LLMs were accessed via their respective commercial APIs (Table 3). No model-specific prompt tuning was performed.

Table 3
LLM configuration.

Setting	Value
Models	Opus-4.6, GPT-5.4, Gemini-3-Flash
Max tokens	16,384
Temperature	0 (override to minimise stochasticity)
Timeout	300 s per call
Prompt format	Markdown with JSON code
Failure handling	JSON extraction failure

Pipeline execution used a single NVIDIA RTX 5060 GPU. Adapted parameter files and CoT reasoning traces were logged for all runs, enabling post-hoc sensitivity analysis (Section 3.4.4).

3.4.3. Evaluation metrics

Segmentation quality is measured primarily by mean Intersection-over-Union (mIoU). For class c ,

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c}, \quad \text{mIoU} = \frac{1}{C} \sum_{c=1}^C \text{IoU}_c, \quad (1)$$

where C is the number of segmental segments (classes) within one ring ($C = 6$ for regular tunnels; $C = 7$ for complex tunnels). As a complementary global metric we also report overall accuracy (OA),

$$\text{OA} = \frac{\sum_{c=1}^C \text{TP}_c}{N}, \quad (2)$$

where N is the total number of points. Because OA can be dominated by majority classes, we treat mIoU as the primary score. We also report per-class IoU to assess whether improvements are broad or concentrated in specific structural classes.

For each condition and LLM, we computed paired per-tunnel improvements as $\Delta_i = \text{mIoU}_{\text{condition},i} - \text{mIoU}_{\text{baseline},i}$, with $n = 3$ for regular tunnels, $n = 2$ for complex tunnels, and $n = 5$ overall. We summarise these paired improvements using three standard statistics: (i) two-sided paired t -test p -values ($\alpha = 0.05$) for statistical significance, (ii) paired Cohen's d for effect size, and (iii) bootstrap 95% confidence intervals (CIs) for the mean improvement.

3.4.4. Sensitivity analysis

To assess the robustness of adapted parameters, for each parameter, we report the *coefficient of variation* (CV):

$$\text{CV} = \frac{s}{|\bar{v}|}, \quad (3)$$

where $\bar{v} = \frac{1}{N} \sum_{i=1}^N v_i$ and $s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (v_i - \bar{v})^2}$ are the mean and standard deviation of the adapted

values $\{v_i\}_{i=1}^N$ across $N = 30$ tunnels. CV measures how much a parameter changes from tunnel to tunnel: (i) a high CV (≥ 0.06) identifies parameters that are *tunnel-responsive*; (ii) $\text{CV} \approx 0$ indicates *baseline corrections* that take the same improved value on every tunnel regardless of geometry. We further examine *cross-LLM consistency*: for parameters that always trigger adaptation, we compare the adapted values, tunnel counts, and tunnel categories produced by each LLM independently.

4. Results

By comparing segmentation performance against the static SAM4Tun baseline and a deterministic non-LLM control, we quantify the contributions of memory (m), state (s), and knowledge (k) across LLMs, identify consistently adapted parameters, and analyse residual errors and runtime trade-offs.

4.1. Overall performance

Table 4 summarises mean mIoU across conditions and LLMs. Under the full m+s+k design, overall mIoU rises from 0.18 (baseline) to 0.43–0.48 across the three LLMs ($p < 0.0001$, paired Cohen's $d = 1.51$ – 1.95); overall OA rises from 0.42 to 0.59–0.65. The improvement holds across both tunnel categories (Fig. 10). Regular tunnels improve from 0.27 to 0.62–0.66 (paired Cohen's $d = 1.4$ – 2.2); regular-tunnel OA rises from 0.50 to 0.77–0.80. Complex tunnels, where the baseline drops to mIoU = 0.04 because several pipeline assumptions no longer match the tunnel conditions, improve to 0.15–0.20 (paired Cohen's $d = 1.2$ – 2.5); complex-tunnel OA rises from 0.30 to 0.33–0.42. Fig. 9 shows the largest improvement observed in each category: regular (staggered), regular (continuous), and complex.

Wall-clock runtime, API-call counts, and per-tunnel input/output token usage with the corresponding indicative USD cost for each LLM under m+s+k are reported in Appendix F (Table 20). Per-class IoU analysis (Appendix G) supports broad improvement across structural classes for regular tunnels and progressive recovery from near-zero baselines for complex tunnels. The full performance distribution is summarised in Appendix H.

Comparison against the non-LLM adaptation.

The non-LLM rule-based adaptation (Table 4, *Non-LLM* column) raises overall mIoU from 0.18 to 0.25 (+0.078, $p = 0.0035$), indicating that part of the gain may reflect deterministic adaptation rather than the LLM-based step alone. The performance distribution varies markedly across tunnel categories: on regular tunnels, the non-LLM adaptation and the static baseline are statistically indistinguishable (≈ 0.27 ; $p = 0.79$), which is likely due to the rule table reproducing the configuration that SAM4Tun was originally tuned for.

On complex tunnels, the rules add +0.113 mIoU ($p = 0.0001$) over the complex category static baseline

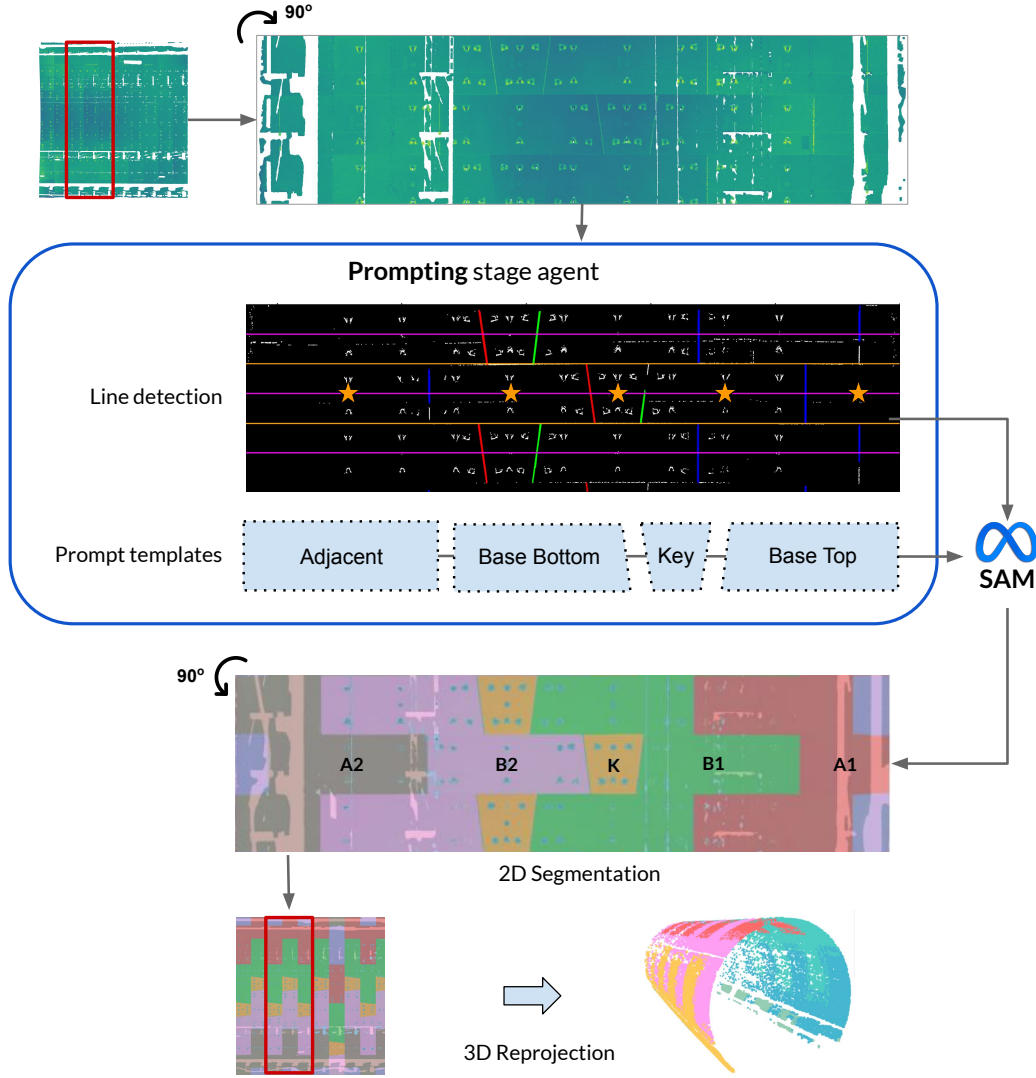


Figure 9: Point-cloud visualisation for the tunnels with the largest mIoU gain in each category: regular (staggered; 2-2), regular (continuous; 3-3-1), and complex (5-5). Top (a): SAM4Tun baseline; bottom (b): LLM-adapted pipeline (m+s+k, Opus-4.6). Colours indicate classes; red denotes error; mIoU is shown per example.

(0.04), by switching tunnel diameter, ring spacing, segment count, and key-segment layout to their large-tunnel specification (i.e., lining-design parameters typically available to on-site engineers). Relative to this rule-based baseline, the LLM (m+s+k) yields a further increase in mean mIoU from 0.150 to 0.15–0.20 (an additional +0.000 to +0.050), improving 2 of 2 complex tunnels. The remaining zero cases occur primarily in tunnels where the deterministic large-tunnel rules already provide a strong prior.

The gap is largest on the regular category, where the non-LLM rule table remains at the SAM4Tun static baseline (regular-baseline mIoU ≈ 0.27), whereas the LLM-based adaptation raises mIoU to 0.62–0.66. Under this experimental setup, the additional gain is therefore consistent with an LLM contribution beyond the deterministic control. On the complex category, both

methods improve over the static baseline (complex-baseline mIoU = 0.04): the rules reach 0.150, while the LLM further improve mIoU to 0.17. Both the LLM and rule-based adaptations converge toward similarly low absolute mIoU under the single-reference design.

4.2. Ablation analysis

To isolate each context component’s contribution, Table 5 reports cumulative mIoU gains at each ablation step. Fig. 10 visualises the step-wise progression for regular and complex categories across all three LLMs.

Memory alone provides an initial reference but is insufficient, and in some settings it degrades performance. On regular tunnels, especially the staggered subsets, memory alone changes mIoU by -0.13 to $+0.08$ across the three LLMs; Table 5; two of the three LLMs show small positive averages, while one shows a small negative average). Without intermediate

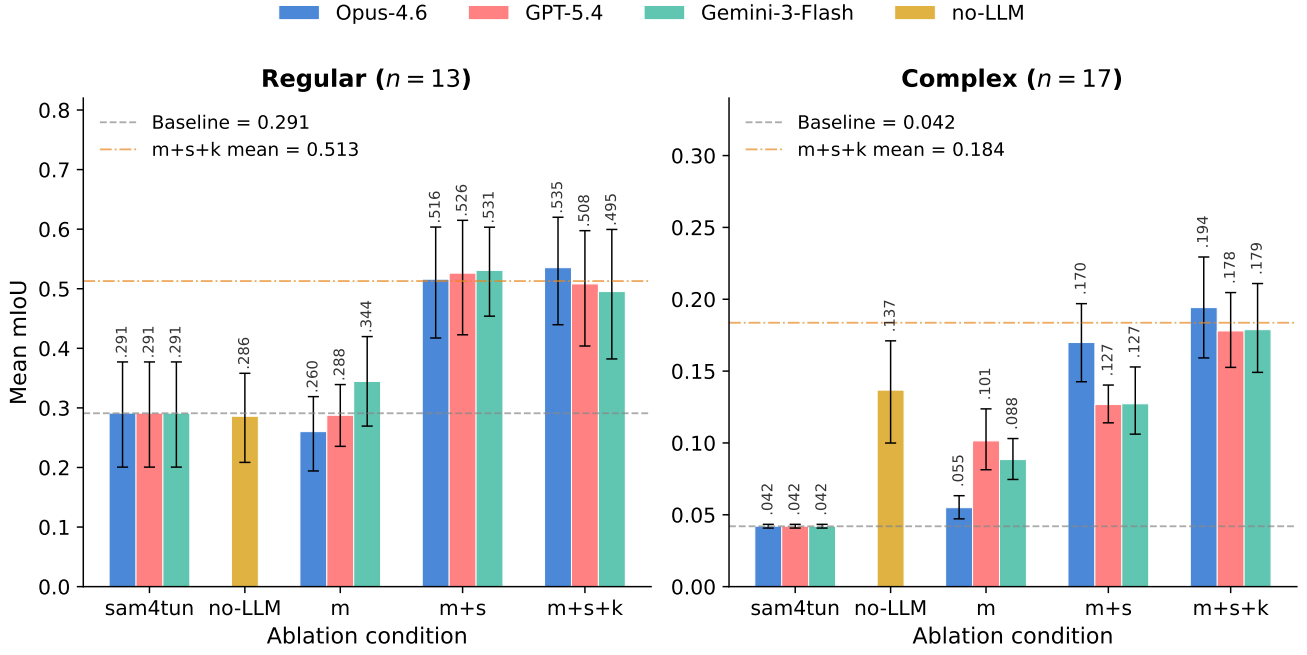


Figure 10: Step-wise adaptation results split by tunnel categories. Bars left to right: SAM4Tun (static baseline), *Non-LLM* (rule-based adaptation), *m*, *m+s*, *m+s+k*. The rule baseline closes part of the gap on complex tunnels but stays flat on regular tunnels; LLM (*m+s+k*) is highest in both categories. Error bars are bootstrap 95% CIs.

Table 4

Overall segmentation results ($n = 5$): mean mIoU and paired Δ mIoU vs. baseline with p -values, effect sizes, and bootstrap 95% CIs. Δ is paired against SAM4Tun. OA is reported as a secondary metric, shown as the first line in each condition segment.

Condition	Statistic	Non-LLM	Opus-4.6	GPT-5.4	Gemini-3-Flash
Baseline (SAM4Tun)	Mean OA	0.419	0.419	0.419	0.419
	Mean mIoU	0.176	0.176	0.176	0.176
Non-LLM	Mean OA	0.475	—	—	—
	Mean mIoU	0.254	—	—	—
	Mean Δ mIoU	+0.078	—	—	—
	p -value	0.0035	—	—	—
	paired Cohen's d	0.58	—	—	—
	std (Δ)	0.051	—	—	—
<i>m</i>	Mean OA	—	0.455	0.476	0.434
	Mean mIoU	—	0.222	0.247	0.220
	Mean Δ mIoU	—	+0.047	+0.071	+0.044
	p -value	—	0.558	0.028	0.056
	paired Cohen's d	—	-0.11	0.42	0.36
	95% CI	—	TBD	TBD	[0.006, 0.101]
<i>m+s</i>	Mean OA	—	0.596	0.576	0.627
	Mean mIoU	—	0.424	0.402	0.465
	Mean Δ mIoU	—	+0.248	+0.227	+0.289
	p -value	—	< 0.0001	< 0.0001	< 0.0001
	paired Cohen's d	—	1.77	1.32	1.46
	95% CI	—	TBD	TBD	TBD
<i>m+s+k</i>	Mean OA	—	0.594	0.623	0.646
	Mean mIoU	—	0.428	0.450	0.479
	Mean Δ mIoU	—	+0.253	+0.274	+0.303
	p -value	—	< 0.0001	< 0.0001	< 0.0001
	paired Cohen's d	—	1.95	1.95	1.51
	95% CI	—	TBD	TBD	TBD

Table 5Cumulative mIoU contribution (mean Δ vs. previous level).

Transition	Opus-4.6	GPT-5.4	Gemini-3-Flash
Baseline $\rightarrow$ <i>Non-LLM</i>	+0.078	+0.078	+0.078
Baseline $\rightarrow$ m	+0.047	+0.071	+0.044
m $\rightarrow$ m+s	+0.202	+0.156	+0.245
m+s $\rightarrow$ m+s+k	+0.004	+0.047	+0.014

feedback, the agent can propose parameter updates but cannot verify whether they improve or degrade the pipeline outputs.

State accounts for the largest observed increment in the ablation study. The m+s condition yields the strongest gain relative to baseline across all three LLMs (all $p < 0.0001$; paired Cohen's $d = 1.32$ – 1.77), while the memory-only effect is small or inconsistent, indicating that state is the main contributor to the observed improvement. State provides the agent with explicit quantitative evidence of how each stage has transformed the data: radial percentiles for mask bounds, retention rates for denoising aggressiveness, coverage uniformity for upsampling targets. Without state, the agent has only pre-pipeline statistics that may not reflect conditions after unfolding, denoising, or enhancing. The same conclusion holds when the comparator is the Non-LLM baseline rather than the static SAM4Tun: state alone (m+s) lifts overall mIoU by 0.15–0.21 above the rule-based adaptation.

One plausible explanation is that the gain associated with state depends on how the model maps raw numeric summaries (percentiles, counts, ratios) to parameter values in light of parameter semantics. We did not test alternative mapping strategies (e.g., regression models); however, the continuous, multidimensional nature of the characteristic space and the non-trivial parameter interactions suggest that capturing this mapping through static rules alone would require considerable manual engineering effort per tunnel category.

Knowledge provides a small additional gain, concentrated in the complex category. On top of m+s, knowledge raises mean mIoU by +0.004 to +0.047 across the three LLMs. The mean increments are small, but the per-tunnel direction is nevertheless positive. The increment is concentrated where the knowledge document supplies specific tuning guidance that neither memory nor state can reliably infer from numerical summaries alone (e.g., a larger-diameter tunnel often uses wider rings).

4.3. Cross-model consistency and repeatability

To assess whether the adaptation behaviour is model-dependent under a fixed prompt and input structure, Table 6 compares the three LLMs on the full m+s+k condition. The three models show similar effect

Table 6

Cross-model summary (m+s+k vs baseline, overall).

LLM	Mean Δ mIoU	95% CI	d
Opus-4.6	+0.253	TBD	1.95
GPT-5.4	+0.274	TBD	1.95
Gemini-3-Flash	+0.303	TBD	1.51

Table 7

Runtime trade-off of the tested R4Tun m+s+k setting.

LLM	Extra time	Total time	Mean Δ mIoU
Gemini-3-Flash	+96 s (0.4 $\times$)	~ 331 s	+0.303
Opus-4.6	+140 s (0.6 $\times$)	~ 375 s	+0.253
GPT-5.4	+307 s (1.3 $\times$)	~ 542 s	+0.274

ranges under the full m+s+k condition, although overlapping 95% confidence intervals do not by themselves rule out between-model differences. A repeatability check under the full m+s+k setting indicates low but non-zero stochastic drift. Across 5 tunnels, each LLM was inferred twice: on average, TBD% of the 18 critical parameters remain unchanged, and the mean absolute inter-run Δ mIoU is TBD (median TBD), as reported in Table 24 (Appendix I).

4.4. Runtime trade-off

R4Tun trades additional API latency for bounded, logged parameter decisions. In this implementation, the SAM4Tun processing takes approximately 235 s per tunnel, and the LLM adaptation under m+s+k adds 96–307 s depending on the LLM, for total times of roughly 331–542 s per tunnel (Table 7). The added runtime is therefore not negligible, but it is modest relative to the cost of annotating data or retraining a supervised model.

4.5. Parameter sensitivity

This analysis separates parameters that vary across tunnels from those that remain constant, clarifying which aspects of the adaptation respond to specific tunnel conditions.

Analysis of 45 adapted parameter runs identified 18 parameters that all three LLMs consistently adjust (Table 8). Of these, 11 are tunnel-responsive ($CV \geq 0.06$), in that their adapted values vary with tunnel geometry, clustering by tunnel category. The remaining 7 parameters behave as baseline corrections ($CV \approx 0$), with near-identical values across tunnels, indicating a shared correction trend relative to the SAM4Tun defaults.

4.6. Error analysis

To understand why the absolute metrics stay low, we sorted every predicted point into four outcomes against the ground truth: *correct*; *false negative* (a

Table 8

Critical parameters identified across all three LLMs (18 total). *Tunnel-responsive* parameters ($CV \geq 0.06$) vary with tunnel geometry; *baseline corrections* ($CV \approx 0$) take near-identical values across tunnels. The meaning/control column briefly explains what each parameter does in the SAM4Tun pipeline.

Stage	Parameter	Meaning/Control	Behaviour	Tunnels	CV	Adapted range / correction
Unfolding	diameter	Cylindrical scale.	Responsive	TBD/5	TBD	TBD
Denoising	mask_r_low	Inner radial gate.	Responsive	TBD/5	TBD	TBD
Denoising	mask_r_high	Outer radial gate.	Responsive	TBD/5	TBD	TBD
Denoising	default_cutoff_z	Fallback radial cutoff (low density).	Responsive	TBD/5	TBD	TBD
Denoising	z_step	Radial histogram bin width.	Responsive	TBD/5	TBD	TBD
Denoising	smooth_win	Cutoff-curve smoothing window.	Correction	TBD/5	TBD	TBD
Denoising	smooth_offset	Post-smoothing cutoff bias.	Correction	TBD/5	TBD	TBD
Denoising	grad_thresh	Outer-edge gradient threshold.	Correction	TBD/5	TBD	TBD
Denoising	y_step	Angular bin width (density filtering).	Correction	TBD/5	TBD	TBD
Enhancing	inter_radius	Interpolation neighbour radius.	Responsive	TBD/5	TBD	TBD
Enhancing	upsampling_stage1	Stage-1 upsampling spacing.	Responsive	TBD/5	TBD	TBD
Enhancing	curv_thresh	Panel-joint curvature threshold.	Correction	TBD/5	TBD	TBD
Enhancing	depth_low	Lower interpolation depth tolerance.	Correction	TBD/5	TBD	TBD
Enhancing	depth_high	Upper interpolation depth tolerance.	Correction	TBD/5	TBD	TBD
Segmenting	hough_thresh_obliq	Oblique joint-line vote threshold.	Responsive	TBD/5	TBD	TBD
Segmenting	hough_thresh_horiz	Horizontal joint-line vote threshold.	Responsive	TBD/5	TBD	TBD
Segmenting	hough_thresh_vert	Vertical ring-boundary vote threshold.	Responsive	TBD/5	TBD	TBD
Segmenting	processing.padding	Segment-crop horizontal padding.	Responsive	TBD/5	TBD	TBD

Table 9

Error composition as a fraction of ground-truth points (Opus-4.6 $m + s + k$ versus SAM4Tun baseline; 3 regular, 2 complex tunnels). FN: segment predicted as background; FP: background predicted as segment; Swap: segment predicted as the wrong segment class. Adaptation reduces false negatives; the residual error is dominated by class swaps.

Category	Method	Correct	FN	FP	Swap
Regular	SAM4Tun $m + s + k$	TBD	TBD	TBD	TBD
		TBD	TBD	TBD	TBD
Complex	SAM4Tun $m + s + k$	TBD	TBD	TBD	TBD
		TBD	TBD	TBD	TBD

segment point left as background, i.e. under-segmentation); *false positive* (a background point pulled into a segment, i.e. over-segmentation, rare here); and *class swap* (a correctly located segment point assigned to the wrong segment type).

Regular and complex tunnels fail differently (Table 9). On regular tunnels, adaptation largely eliminates under-segmentation, reducing false negatives from TBD% to TBD% of ground-truth points. On complex tunnels, adaptation recovers most under-segmented blocks (false negatives TBD% to TBD%). For both categories, most remaining mistakes are class swaps. Swapped labels persist even when the correct points are recovered (TBD% to TBD% on regular tunnels; rising to TBD% on complex tunnels). In both categories, therefore, the remaining error is a labelling problem, not a detection or noise problem. Fig. 12 shows the same outcomes spatially: after adaptation, false negatives (blue) and false positives (orange) are sparse, whereas class swaps (red) account for most residual error, especially on the complex tunnel.

The failure modes trace back to a mismatch between how the SAM4Tun pipeline is parameterised and how segmental rings actually vary. The pipeline applies *one parameter set per tunnel* and labels each ring by first

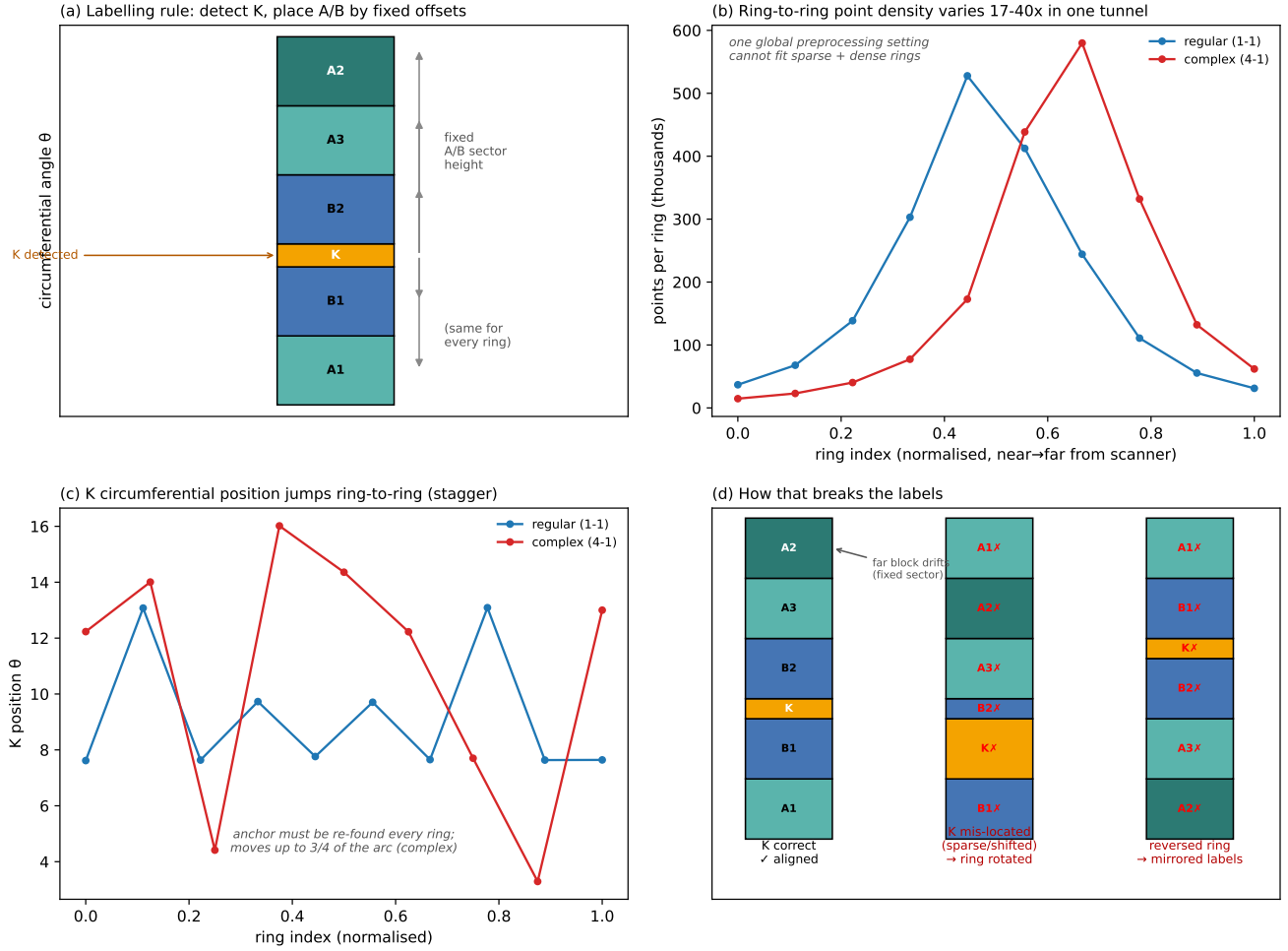


Figure 11: Structural constraints of the fixed SAM4Tun labelling rule. (a) Labelling rule: after K is detected, B segments are placed one step either side of K and A segments at successive multiples of the same fixed offset, in a fixed order, so the template mislabels rings whose segment widths or count differ from the reference. (b) Ground-truth K position versus ring index for one regular (blue, 2-2) and one complex (red, 5-5) tunnel: K jumps from ring to ring, much more on the complex tunnel, which the current one-configuration-per-tunnel setting does not address.

detecting the key segment (K) and then placing every other segment at a *fixed* angular offset from K, in a *fixed* order. Neither of structural constraints can be removed by tunnel-level parameter tuning.

1. *Per-ring density variation under tunnel-level parameterisation.* R4Tun adapts one configuration per tunnel, but point density is far from uniform *within* a tunnel: the per-ring point count varies by up to 19-fold (regular) and 38-fold (complex) between the dense central rings and the sparse end rings. A single tunnel-level setting cannot keep both ends of this range inside the detector's working band, so the sparse rings systematically lose ring-boundary and K detection no matter how the tunnel-level parameters are chosen. This is a limitation of the adaptation *scope* (one configuration per tunnel), not of any individual parameter value, and it can only be lifted by moving from tunnel-level to ring-level adaptation.

2. *Fixed segment-offset template.* Once K is located, the rule places the B segments one step on either side of K and the A segments at successive multiples of that same fixed step, in a fixed order (Fig. 11a). The step size and the segment order are hard-coded, so when the true geometry departs from the reference the fixed steps no longer line up with the real joints and correctly detected points are still given the wrong label. This is the single largest source of class swaps, visible as the red bands in the right column of Fig. 12, which grow from 10% (regular) to 42% (complex) even where points are correctly located.

Lifting these limits is future work rather than a parameter change, because they are baked into the one-configuration-per-tunnel, detect-K-then-fixed-template logic of SAM4Tun. The clearest direction is to move

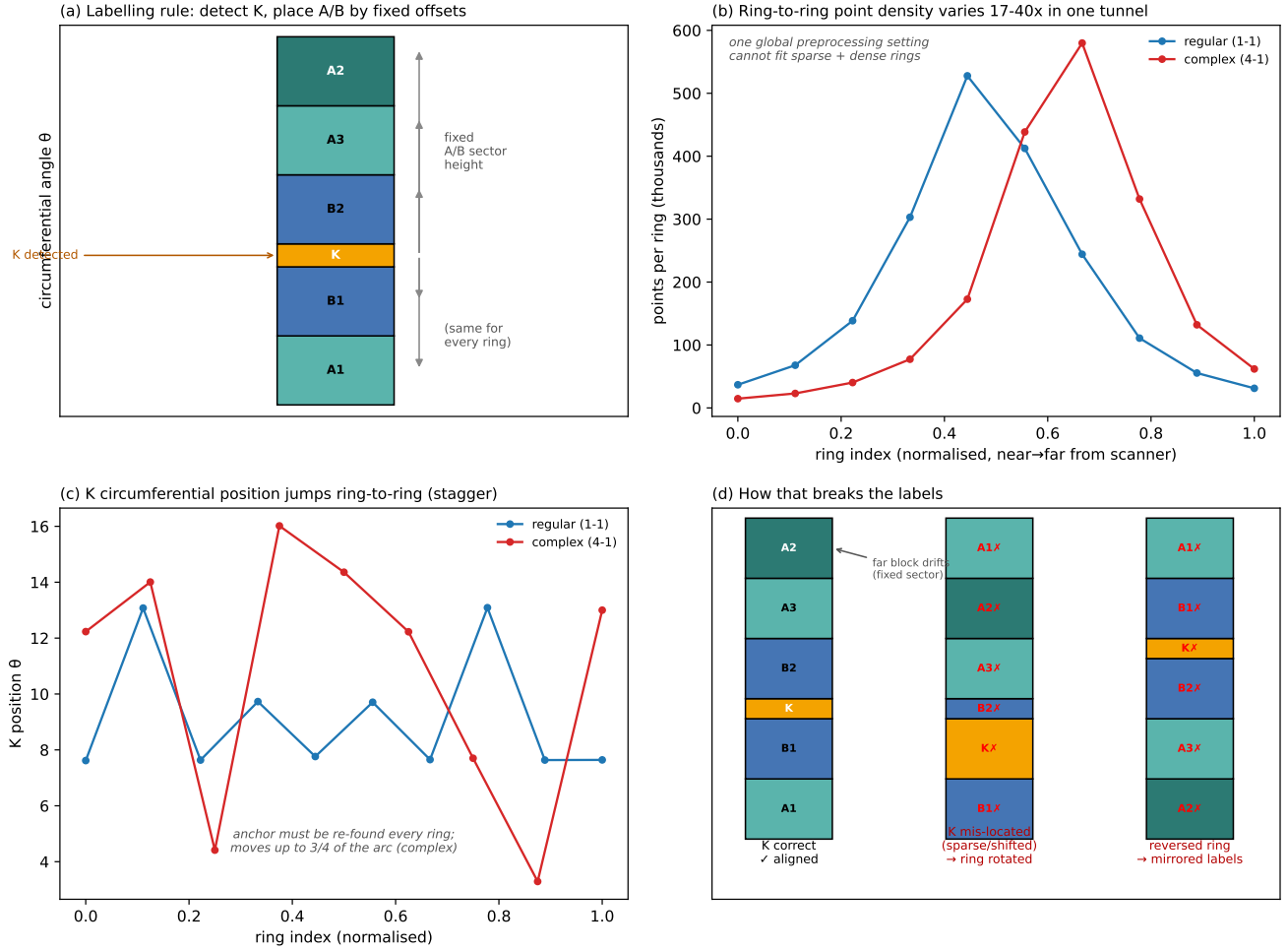


Figure 12: Per-point error maps for a representative regular tunnel (a, 2-2) and complex tunnel (b, 5-5) under $m + s + k$ (Opus-4.6), drawn on the unfolded surface (x: ring index; y: circumferential angle). **Left column**—detection outcomes: correct (grey), false negatives (blue, segment→background) and false positives (orange, background→segment), with the ground-truth K (green) and predicted K (red) overlaid per ring; titles give FN, FP, and mIoU. **Right column**—class swaps (red) among otherwise correctly located points. On the regular tunnel the predicted K stays on the GT K and swaps remain low (TBD%). On the complex tunnel the predicted K is nearly constant and drifts away from the GT K, which jumps ring-to-ring; each affected ring's labels then rotate together and swaps rise to TBD%. The maps localise the two constraints: the K mismatch (left) drives the moving-anchor errors, and the resulting label rotation (right) is the dominant residual error.

from tunnel-level to ring-level adaptation: density-adaptive preprocessing and per-ring K re-anchoring would target the density and K-detection problems above, while a variable-length segment template and handedness inference would relax the fixed labelling rule, together replacing it with dynamic, context-aware labelling. Implementing this would require rewriting the current open-source implementation, which lies outside the bounded-parameter adaptation studied here. These directions extend SAM4Tun and R4Tun rather than invalidate them: the present study establishes that structured context lets an LLM adapt parameters within fixed bounds, and pinpoints which structural parts of the underlying pipeline must change next to lift the absolute ceiling.

5. Discussion

Taken together, the results support a mechanism-level claim rather than a deployment-level one. With the SAM4Tun pipeline and the expert reference held fixed, structured context improves bounded parameter adaptation across three LLMs. In practical terms, $m + s + k$ can guide a reviewable parameter update without labelled retraining; however, at current accuracy it supports assisted analysis, not autonomous inspection.

5.1. Key findings

The experimental evaluation yields two primary findings. First, as noted in the ablation study (Section 4.2), the ablation results suggest that the context design improves adaptation across models and tunnel

categories, with state providing the main gains (+0.156 to +0.245 mIoU on top of memory). Second, the LLM (m+s+k) gain over the non-LLM rule-based adaptation is concentrated on the regular category, where the rule table cannot improve over the static baseline (Section 4.1). This concentration is consistent with the single-reference design: both comparators start from a configuration tuned on a tunnel that is similar to the regular category. Near the reference, the LLM has more informative context to condition on, and its advantage over deterministic lookup becomes visible; far from the reference (the complex category), neither approach has a matching anchor and the two converge toward similarly low absolute performance, with the LLMs retaining a small mean advantage. As reported in Section 4.1, the regular category gap occurs where the deterministic control does not improve over the static baseline, whereas the complex category convergence more likely reflects a shared ceiling than a specific failure of LLM reasoning (Section 5.3).

In practice, a domain expert calibrates the pipeline on a single reference tunnel, encodes the tuning rationale into structured documents, and deploys R4Tun for subsequent tunnels. Under this single-reference setup, the framework is best suited to targets that remain close to the calibrated reference, and it can also flag when the fixed pipeline begins to fail. The framework supports post-hoc review by logging a rationale alongside each parameter change, and requires no model retraining or labelled domain data. Experts must still define the reference configuration and bounds, review rationales, and judge whether low-confidence outputs are acceptable. Within this scope, R4Tun does not yet support autonomous final inspection or downstream tasks (e.g. structural health monitoring or Scan-to-BIM) without human verification.

5.2. Comparison with state-of-the-art

We position R4Tun against established tunnel-segmentation paradigms in Table 10. The comparison is presented as an operating-point and setup-cost comparison rather than a direct accuracy ranking, because the methods are not evaluated under comparable conditions. Supervised 3D deep-learning models on Seg2Tunnel (LiningNet, SparseUNet, SCF-Net, FA-ResNet) report mIoU 0.83–0.90 [?], but are evaluated in-distribution and require per-point labels and GPU retraining. Geometric feature-engineering methods avoid training but depend on manually tuned thresholds and typically report cross-section fitting error rather than per-segment mIoU, so they are not directly comparable at the component level. Expert-tuned SAM4Tun reaches mIoU above 0.90, but only on curated, per-case-tuned test rings [12]; under our protocol (5 diverse subsets, single fixed reference, no per-case tuning) the same open-source pipeline reaches

only 0.531 on its own reference tunnel and 0.18 static across all subsets.

The only directly comparable pair (same evaluation protocol, with no labels, no training, and no per-case tuning) is R4Tun (0.43–0.48) versus static SAM4Tun (0.18); R4Tun improves the directly comparable baseline. The supervised and expert-tuned figures are obtained in more favourable settings and are reported here to bound the remaining accuracy gap honestly, not as direct competitors. Deterministic, non-LLM rule baseline (Section 4.1) reaches only 0.25, and every higher figure in Table 10 requires labels, training, or per-case expert tuning.

5.3. Limitations

The limitations of this work are primarily driven by the fixed SAM4Tun operator design and by the single-reference adaptation protocol. In particular, R4Tun inherits SAM4Tun’s assumptions and failure modes—especially its fixed offset-and-order templates—so the LLM can only re-parameterise the pipeline rather than replace it. All parameter adaptation is anchored to a single expert-tuned configuration, which creates a low ceiling for both the LLM-guided and the rule-based methods. When no contextual anchor resembles the target tunnel, neither approach has sufficient information to recover strong performance. Expanding R4Tun to encode multiple expert-tuned reference configurations (e.g., one per tunnel category) may further narrow the absolute gap.

Our study was evaluated exclusively on the SAM4Tun pipeline and the Seg2Tunnel dataset; consequently, its transferability is argued conceptually rather than validated empirically, and is most plausible when four conditions hold: (i) each stage exposes bounded, tunable parameters (i.e., adaptation is a re-parameterisation problem rather than a redesign); (ii) expert reference configurations exist whose target characterisation is comparable to the new deployment setting; (iii) each stage admits a compact, summarised intermediate state that can be passed between agents; and (iv) stage-specific knowledge can be encoded (e.g., as bounds, rules, and failure signatures) for use during adaptation.

6. Conclusions

Reliable segmentation of segmental tunnel linings from point clouds remains challenging when tunnel conditions diverge from the expert-tuned reference. This paper presented R4Tun, a multi-agent framework that extends a fixed segmentation pipeline with LLM-guided parameter adaptation. Rather than replacing the underlying geometric operators, R4Tun adapts their parameters by comparing each new tunnel with the reference configuration and by using compact summaries of intermediate pipeline outputs. This design preserves the deterministic structure of the original

Table 10

Positioning R4Tun against representative tunnel-segmentation paradigms. The comparison is on setup cost as well as accuracy: ✓ indicates the requirement (or capability) applies, “–” that it does not. mIoU figures are *not* measured under comparable settings (see the “setting” column): supervised and expert-tuned figures are taken from [?] and [12] under their original, more favourable protocols, whereas the static-SAM4Tun, non-LLM, and R4Tun figures are from this study under a single fixed reference across 5 diverse subsets. R4Tun is the only approach that is simultaneously label-free, training-free, and expert-tuning-free while still adapting per tunnel.

Method	Labels	Training	Per-case expert tuning	Per-tunnel adaptation	mIoU (setting)
LiningNet/SparseUNet/SCF-Net/FA-ResNet	✓	✓	–	–	0.83–0.90 (in-distribution)
Expert-tuned SAM4Tun	–	–	✓	✓ (manual)	above 0.90 (curated rings)
Static SAM4Tun (this protocol)	–	–	–	–	0.18
Non-LLM rules (this study)	–	–	–	✓	0.25
R4Tun (this study)	–	–	–	✓ (auto)	0.43–0.48

pipeline while supporting post-hoc review and revision of parameter changes.

Evaluated on 5 subsets spanning 3 regular and 2 complex tunnels with three different LLMs, the full $m + s + k$ design raised mean mIoU from 0.18 to 0.43–0.48 and OA from 0.42 to 0.59–0.65, with per-class IoU improving broadly across structural classes. Regular tunnels improve markedly, with mIoU increasing from 0.27 to 0.62–0.66 and OA increasing from 0.50 to 0.77–0.80. Complex tunnels recover from near-zero baselines, with mIoU increasing from 0.04 to 0.15–0.20, and OA from 0.30 to 0.33–0.42. The ablation study suggested that state is an important contributor to the improvement: adding state increases overall mIoU by 0.15–0.21 from the static baseline 0.18.

In the regular category, the non-LLM rule-based adaptation matches the static baseline (mIoU $\approx$ 0.27), whereas the LLM raises mIoU to 0.62–0.66. This result is therefore consistent with an additional LLM-related contribution. In the complex category, both the LLM and rule-based adaptations converge to low absolute mIoU, which we attribute to the single-reference design rather than to LLM reasoning quality.

For construction practice, R4Tun may shift expert effort from repeated per-tunnel intervention toward an upfront authoring step (reference calibration plus per-stage knowledge documents). It logs a rationale alongside each parameter change to support review, and it can be executed via commercial LLM APIs under the tested settings without retraining on labelled domain data.

Despite these improvements, the current absolute accuracy is not yet sufficient to support autonomous final inspection or downstream tasks without human verification. The main limitations are as follows. First, all parameter adaptation is anchored to a single expert-tuned reference configuration, which sets a low ceiling for both the LLM-guided and rule-based methods; when the target tunnel is far from this anchor (e.g., the complex category), neither approach has a closely

matching reference from which to recover strong performance. Second, R4Tun inherits the assumptions and failure modes of the underlying SAM4Tun operators—in particular the fixed offset-and-order labelling template—so the LLM can only re-parameterise the pipeline; residual class-swap errors and the absolute-accuracy ceiling therefore persist regardless of parameter choice.

Several directions for future work follow from these limitations. First, replacing SAM4Tun’s fixed-template labelling logic with a dynamic, context-aware alternative could raise the absolute performance ceiling. Second, encoding multiple expert-designed reference configurations could improve adaptation to atypical tunnels. Third, validation on broader tunnel typologies and acquisition conditions would strengthen generalisability claims.

Data availability

The source code, adapted parameters, and evaluation scripts are available at <https://github.com/Tao-Robominds/R4Tun>. The Seg2Tunnel dataset is publicly available.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI use

During this study, the authors evaluated large language models (Opus-4.6, GPT-5.4, and Gemini-3-Flash) as the main experimental systems. The LLMs were also used for language editing and for improving manuscript structure. The authors reviewed and edited all content and take full responsibility for the content of the publication.

References

- [1] L. Attard, C. J. Debono, G. Valentino, M. D. Castro, Tunnel inspection using photogrammetric techniques and image processing: A review, *ISPRS Journal of Photogrammetry and Remote Sensing* 144 (2018) 180–188. doi:10.1016/j.isprsjprs.2018.07.004.
- [2] L. Weidner, G. Walton, Generalized extraction of bolts, mesh, and rock in tunnel point clouds: a critical comparison of geometric feature-based methods using random forest and neural networks, *Remote Sensing* 16 (2024) 678. doi:10.3390/rs16040678.
- [3] M. Q. Huang, J. NiniÄĖ, Q. B. Zhang, Bim, machine learning and computer vision techniques in underground construction: current status and future perspectives, *Tunnelling and Underground Space Technology* 108 (2021) 103677. doi:10.1016/j.tust.2020.103677.
- [4] A. Sjölander, V. Belloni, A. Ansell, E. Nordström, Towards automated inspections of tunnels: a review of optical inspections and autonomous assessment of concrete tunnel linings, *Sensors* 23 (2023) 5457. doi:10.3390/s23125457.
- [5] R. Montero, J. G. Victores, S. MartÄĖnez, A. JardÄĖn, C. Balaguer, Past, present and future of robotic tunnel inspection, *Automation in Construction* 59 (2015) 99–112. doi:10.1016/j.autcon.2015.07.001.
- [6] A. Strauss, J. Bien, H. Neuner, C. Harmening, C. Seywald, M. ÄĖsterreicher, K. Voit, E. Pistone, P. Spyridis, K. Bergmeister, Sensing and monitoring in tunnels: testing and monitoring methods for the assessment of tunnels, *Structural Concrete* 21 (2020) 1234–1248. doi:10.1002/suco.201900456.
- [7] Y. Duan, S. Qiu, W. Jin, T. Lu, X. Li, High-speed rail tunnel panoramic inspection image recognition technology based on improved yolov5, *Sensors* 23 (2023) 6786. doi:10.3390/s23156786.
- [8] Y. J. Cha, R. Ali, J. Lewis, O. Buyukozturk, Deep learning-based structural health monitoring, *Automation in Construction* 161 (2024) 105324. doi:10.1016/j.autcon.2024.105324.
- [9] C. Su, Q. Hu, Z. Yang, R. Huo, A review of deep learning applications in tunneling and underground engineering in china, *Applied Sciences* 14 (2024) 3234. doi:10.3390/app14083234.
- [10] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, E. Brynjolfsson, S. Buch, D. Card, R. Castellon, N. Chatterji, A. Chen, K. Creel, J. Q. Davis, D. Demszky, C. Donahue, M. Doumbouya, E. Durmus, S. Ermon, J. Etchemendy, K. Ethayarajh, L. Fei-Fei, C. Finn, T. Gale, K. Goel, N. Goodman, S. Grossman, et al., On the opportunities and risks of foundation models, *arXiv preprint arXiv:2108.07258* (2021). URL: <https://arxiv.org/abs/2108.07258>.
- [11] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. DollÄĖr, R. Girshick, Segment anything, *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (2023) 3992–4003.
- [12] Z. Ye, W. Lin, A. Faramarzi, X. Xie, J. NiniÄĖ, Sam4tun: No-training model for tunnel lining point cloud component segmentation, *Tunnelling and Underground Space Technology* 158 (2025) 106401. doi:10.1016/j.tust.2025.106401.
- [13] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, M. Sun, Chatdev: Communicative agents for software development, *arXiv preprint arXiv:2307.07924* (2024). URL: <https://arxiv.org/abs/2307.07924>.
- [14] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, J. Schmidhuber, Metagpt: Meta programming for a multi-agent collaborative framework, *arXiv preprint arXiv:2308.00352* (2024). URL: <https://arxiv.org/abs/2308.00352>.
- [15] P. Chen, B. Han, S. Zhang, Comm: Collaborative multi-agent, multi-reasoning-path prompting for complex problem solving, *arXiv preprint arXiv:2405.00847* (2024). URL: <https://arxiv.org/abs/2405.00847>.
- [16] C. I. Garcia, M. A. Dibattista, T. A. Letelier, H. D. Halloran, J. A. Camelio, Framework for llm applications in manufacturing, *Manufacturing Letters* 40 (2024) 56–63. doi:10.1016/j.mfglet.2024.08.003.
- [17] M. A. Fischler, R. C. Bolles, Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography, *Communications of the ACM* 24 (1981) 381–395.
- [18] M. Ester, H. P. Kriegel, J. Sander, X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)* (1996) 226–231.
- [19] M. Pauly, M. Gross, L. P. Kobbelt, Efficient simplification of point-sampled surfaces, *Proceedings of IEEE Visualization* (2002) 163–170.
- [20] C. R. Qi, L. Yi, H. Su, L. J. Guibas, Pointnet++: Deep hierarchical feature learning on point sets in a metric space, in: *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL: <https://arxiv.org/abs/1706.02413>.
- [21] Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, A. Markham, Randla-net: Efficient semantic segmentation of large-scale point clouds, *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020). URL: <https://arxiv.org/abs/1911.11236>.
- [22] J. Schult, F. Engelmann, A. Hermans, O. Litany, S. Tang, B. Leibe, Mask3d: Mask transformer for 3d semantic instance segmentation, *arXiv preprint arXiv:2303.05475* (2023). URL: <https://arxiv.org/abs/2303.05475>.
- [23] M. Kolodiazny, A. Vorontsova, A. Konushin, D. Rukhovich, Top-down beats bottom-up in 3d instance segmentation, *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* (2024) 3566–3574.
- [24] E. Camuffo, D. Mari, S. Milani, Recent advancements in learning algorithms for point clouds: an updated overview, *Sensors* 22 (2022) 1357. doi:10.3390/s22041357.
- [25] M. Liu, R. Shi, K. Kuang, Y. Zhu, X. Li, S. Han, H. Cai, F. Porikli, H. Su, OpenShape: Scaling up 3d shape representation towards open-world understanding, in: *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [26] Z. Guo, R. Zhang, X. Zhu, Y. Tang, X. Ma, J. Han, K. Chen, P. Gao, X. Li, H. Li, P.-A. Heng, Point-bind & point-llm: Aligning point cloud with multi-modality for 3d understanding, generation, and instruction following, *arXiv preprint arXiv:2309.00615* (2023).
- [27] R. Xu, X. Wang, T. Wang, Y. Chen, J. Pang, D. Lin, Pointllm: Empowering large language models to understand point clouds, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024, pp. 131–147.
- [28] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, A. Joulin, Emerging properties in self-supervised vision transformers, *arXiv* (2021). doi:10.48550/arXiv.2104.14294. arXiv:2104.14294, version 2, revised 24 May 2021.
- [29] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. RÄĖdle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. DollÄĖr, C. Feichtenhofer, Sam 2: Segment anything in

- images and videos, arXiv preprint arXiv:2408.00714 (2024). URL: <https://arxiv.org/abs/2408.00714>.
- [30] X. Zou, J. Yang, H. Zhang, F. Li, L. Li, J. Wang, L. Wang, J. Gao, Y. J. Lee, Segment everything everywhere all at once, arXiv preprint arXiv:2304.06718 (2023). URL: <https://arxiv.org/abs/2304.06718>.
- [31] B. Wang, Z. Chen, M. Li, Q. Wang, C. Yin, J. C. P. Cheng, Omni-scan2bim: A ready-to-use scan2bim approach based on vision foundation models for mep scenes, Automation in Construction 167 (2024) 105678. doi:10.1016/j.autcon.2024.105678.
- [32] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, R. Lowe, Training language models to follow instructions with human feedback, arXiv preprint arXiv:2203.02155 (2022). URL: <https://arxiv.org/abs/2203.02155>.
- [33] DeepSeek-AI, DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, arXiv preprint arXiv:2501.12948 (2025). URL: <https://arxiv.org/abs/2501.12948>. doi:10.48550/arXiv.2501.12948. arXiv:2501.12948.
- [34] J. Chua, O. Evans, Are deepseek r1 and other reasoning models more faithful?, arXiv preprint arXiv:2501.07632 (2025). URL: <https://arxiv.org/abs/2501.07632>.
- [35] V. Xiang, C. Snell, K. Gandhi, A. Albalak, A. Singh, C. Blagden, D. Phung, R. Rafailov, N. Lile, D. Mahan, L. Castricato, J.-P. Fränken, N. Haber, C. Finn, Towards System 2 reasoning in LLMs: Learning how to think with meta chain-of-thought, arXiv preprint arXiv:2501.04682 (2025). URL: <https://arxiv.org/abs/2501.04682>. doi:10.48550/arXiv.2501.04682. arXiv:2501.04682.
- [36] OpenAI, OpenAI o3 and o4-mini system card, <https://openai.com/index/o3-o4-mini-system-card/>, 2025. Accessed: 2025-12-01.
- [37] OpenAI, Reasoning best practices, <https://platform.openai.com/docs/guides/reasoning-best-practices>, 2025. Accessed: 2025-12-01.
- [38] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, arXiv preprint arXiv:2201.11903 (2023). URL: <https://arxiv.org/abs/2201.11903>.
- [39] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, Y. Iwasawa, Large language models are zero-shot reasoners, arXiv preprint arXiv:2205.11916 (2023). URL: <https://arxiv.org/abs/2205.11916>.
- [40] Z. Zhang, Y. Yao, A. Zhang, X. Tang, X. Ma, Z. He, Y. Wang, M. Gerstein, R. Wang, G. Liu, H. Zhao, Igniting language intelligence: the hitchhiker's guide from chain-of-thought reasoning to language agents, arXiv preprint arXiv:2311.11797 (2023). URL: <https://arxiv.org/abs/2311.11797>.
- [41] Y. Zhang, R. Sun, Y. Chen, T. Pfister, R. Zhang, S. A. Arik, Chain of agents: Large language models collaborating on long-context tasks, arXiv preprint arXiv:2406.02818 (2024). URL: <https://arxiv.org/abs/2406.02818>.
- [42] P. Xu, W. Ping, X. Wu, L. McAfee, C. Zhu, Z. Liu, S. Subramanian, E. Bakhturina, M. Shoeybi, B. Catanzaro, Retrieval meets long context large language models, arXiv preprint arXiv:2310.03025 (2024). URL: <https://arxiv.org/abs/2310.03025>.
- [43] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, S. Liu, A survey of context engineering for large language models, arXiv preprint arXiv:2501.04567 (2025). URL: <https://arxiv.org/abs/2501.04567>.
- [44] Anthropic, Effective context engineering for ai agents, <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>, 2025. Accessed: 2025-12-01.
- [45] R. O. Duda, P. E. Hart, Use of the hough transformation to detect lines and curves in pictures, Communications of the ACM 15 (1972) 11–15.

Appendices

A. Baseline parameter tables

Tables 11–14 report the SAM4Tun baseline parameter values used as the reference configuration for all adaptation experiments.

B. Characteriser fields

Table 15 summarises the characteriser fields used to describe each tunnel and to populate the per-stage state provided to the agents.

C. Context components: denoising agent example

Appendices C and D share a single worked example: the *denoising* agent applied to tunnel 4-1 (m+s+k condition, Opus 4.6). This appendix reproduces the three context components (memory, state, knowledge) that the agent receives as input; Appendix D reproduces the CoT trace and adapted-parameter JSON it returns.

C.1. Memory: reference vs target raw characteristics

Memory pairs the reference tunnel’s raw characteristics with those of the target tunnel using an identical schema, so the agent can compute deviations field-by-field (Table 16).

Memory also bundles the SAM4Tun reference parameters for the stage (Table 12), so the agent always has a known-good baseline to deviate from.

C.2. State: cumulative outputs of upstream stages

State grows as the pipeline executes. For the denoising agent, state consists of the unfolded characteristics produced by the preceding unfolding stage, supplied as a reference/target pair (Table 17).

C.3. Knowledge: parameter semantics, ranges, and constraints

Knowledge is a stage-specific Markdown document, authored once and shared across all tunnels. It enumerates each parameter together with its empirically validated range, defaults, and inter-parameter constraints. The denoising knowledge document is reproduced verbatim below.

Tunable parameters. *mask_r_low* (m), inner radial gate before depth histogramming, range [2.09, 3.75] (baseline 2.7); *mask_r_high* (m), outer radial gate, range [2.78, 4.38] (baseline 2.8); *default_cutoff_z* (m), fallback radial cut off when a θ -bin lacks reliable counts, range [2.65, 6.27] (baseline 2.7); *z_step* (m), radial bin width per histogram column, range [0.003, 0.005] (baseline 0.001).

Table 11

Unfolding parameters (sam4tun baseline).

Parameter	Value
<i>delta</i>	0.005
<i>slice_spacing_factor</i>	1.2
<i>vertical_filter_window</i>	4.5
<i>ransac_threshold</i>	1
<i>ransac_probability</i>	0.9
<i>ransac_inlier_ratio</i>	0.75
<i>ransac_sample_size</i>	5
<i>polynomial_degree</i>	3
<i>num_samples_factor</i>	1210
<i>diameter</i>	5.5

Table 12

Denoising parameters (sam4tun baseline).

Parameter	Value
<i>mask_r_low</i>	2.7
<i>mask_r_high</i>	2.8
<i>y_step</i>	0.5
<i>z_step</i>	0.001
<i>grad_threshold</i>	0.2
<i>smoothing_window_size</i>	3
<i>smoothing_offset</i>	−0.003
<i>default_cutoff_z</i>	2.7

Table 13

Enhancing parameters (sam4tun baseline).

Parameter	Value
<i>upsamp_stage1_target_dist</i>	0.08
<i>upsamp_stage2_target_dist</i>	0.04
<i>upsamp_stage3_target_dist</i>	0.02
<i>curvature_threshold</i>	0.0005
<i>depth_threshold_low</i>	0.003
<i>depth_threshold_high</i>	0.01
<i>inter_radius</i>	0.06
<i>duplicate_threshold</i>	0.02
<i>num_neighbors</i>	20
<i>num_interpolations</i>	2
<i>resolution</i>	0.005
<i>window_size</i>	9

Proven defaults (fixed, not searched).

smoothing_window_size = 5, *smoothing_offset* = −0.002, *grad_threshold* = 0.15, *y_step* = 0.4.

Locked parameters. All other keys in the stage JSON remain at the pipeline defaults reported in Table 12.

Constraints. *mask_r_low* < *mask_r_high*; *default_cutoff_z* should stay coherent with the upstream unfolding diameter (order of magnitude $\approx$ diameter/2, with diameter in metres).

The four other stages (unfolding, enhancing, detecting, segmenting) follow the same four-block template;

Table 14

Detecting parameters (sam4tun baseline).

Parameter	Value
binary_threshold	127
morph_kernel_size	[3, 3]
dilation_iterations	1
hough_thresh_oblique	50
minLineLength_oblique	100
maxLineGap_oblique	40
hough_thresh_horiz	50
minLineLength_horiz	100
maxLineGap_horiz	10
hough_thresh_vert	500
angle_range_obliq_pos	[6, 9]
angle_range_obliq_neg	[-9, -6]
merge_distance	3
ring_spacing_constant	1.2
resolution	0.005

Table 15

Characteriser fields by stage. Raw fields are available to all stages; each subsequent group becomes available after the corresponding stage completes.

Source	Fields
Raw	estimated diameter, tunnel length, tunnel height z-range (min, max) mean / median / min nearest-neighbour distance
Unfolded	r-percentiles (p_{10} , p_{99}), h-span, θ -span, θ -range median / std nearest-neighbour distance intensity median, intensity min
Denoised	mean / median / std nearest-neighbour distance estimated diameter, tunnel length, surface completeness surface regularity, average curvature, section curvatures
Enhanced	total points, median / mean nearest-neighbour distance coverage uniformity template spacing suitability, current median spacing

the per-stage knowledge documents are released in the project's code repository.

D. Worked CoT trace

The following trace continues the worked example from Appendix C: the denoising agent receives the memory, state, and knowledge inputs above, executes the five-step CoT protocol (Section 3.3.2), and emits a single schema-conformant JSON object that the denoising stage consumes unchanged.

Table 16

Memory excerpt (raw characteristics). Reference is the expert-tuned tunnel; target is tunnel 4-1.

Field	Reference	Target (4-1)	Δ
Estimated diameter (m)	5.50	7.50	+36%
Tunnel length (m)	12.16	—	—
Tunnel height (m)	5.08	—	—
Mean NN distance (m)	0.0082	0.0114	+39%
Median NN distance (m)	0.0065	0.0089	+37%
Point density (pts/m ³)	2,466	1,842	-25%

Table 17

State excerpt (unfolded characteristics, after the unfolding stage).

Field	Reference	Target (4-1)
r-percentile p_{10} (m)	2.30	2.63
r-percentile p_{99} (m)	2.77	4.02
h-span (m)	12.08	6.08
θ -span (rad)	17.28	7.65
Median NN distance (m)	0.0042	0.0086

Step 1 — Referencing. Diameter is 7.5 m vs reference 5.5 m (+36%). Unfolded r-percentiles are $p_{10} = 2.63$, $p_{99} = 4.02$, vs reference $p_{10} = 2.30$, $p_{99} = 2.77$. Median nearest-neighbour distance is 0.0086 m vs reference 0.0042 m, indicating point density at $\sim 50\%$ of the reference.

Step 2 — Diagnostic inspection. Large-diameter, moderate-sparse target. The reference mask_r_low/mask_r_high (2.7/2.8) bracket only the reference p_{99} and would clip $\sim 60\%$ of wall points at the target radius; the radial gate must widen. default_cutoff_z (2.7) is below the target p_{99} (4.02) and would force the fallback path to discard valid wall points; it must rise. Sparse density requires z_step to relax from 0.001 so each radial bin retains enough samples for the gradient test.

Step 3 — Parameter adaptation. Set mask_r_low = 2.37 (just below target p_{10}); mask_r_high = 3.80 (above target $p_{99} = 4.02$ would exceed the empirical range [2.78, 4.38] when widened further; 3.80 is the largest value that retains $\geq 99\%$ of wall points while staying within range); default_cutoff_z = 3.7 (above mask_r_high and order $\approx$ diameter/2); z_step = 0.005 (proven default for sparse scans). All other parameters take the proven defaults from the knowledge block.

Step 4 — Validation. mask_r_low (2.37) < mask_r_high (3.80); both lie within their empirical ranges. default_cutoff_z (3.7) is consistent with the unfolding diameter (7.5 m, half-diameter 3.75 m). z_step (0.005) lies in [0.003, 0.005]. All keys present, all types match Table 12.

Step 5 — JSON output.

Table 18

Worked example output (denoising agent, tunnel 4-1, m+s+k, Opus 4.6).

Parameter	Baseline	Adapted	Rationale
mask_r_low	2.7	2.37	Just below target p_{10}
mask_r_high	2.8	3.80	Covers target p_{90}
default_cutoff_z	2.7	3.7	$\approx \text{diameter}/2$
z_step	0.001	0.005	Sparse-density
smoothing_window_size	3	5	Proven default
smoothing_offset	-0.003	-0.002	Proven default
grad_threshold	0.2	0.15	Proven default
y_step	0.5	0.4	Proven default

Table 19

On-site rules baseline: inputs and rule-driven parameters.

Stage	Parameter	Rule
Unfolding	diameter	$= d_{\text{measured}}$
Denoising	mask_r_low	$= d/2 - 0.15$
Denoising	mask_r_high	$= d/2 + 0.15$
Denoising	default_cutoff_z	$= \text{mask_r_high} + 0.05$
Segmenting	ring_spacing_constant	$= L_{\text{ring}}$ (1.2 or 1.8 m)
Segmenting	segment_per_ring	$= 6$ (T1–T3) or 7 (T4–T5)

```
{
  "mask_r_low": 2.37,
  "mask_r_high": 3.80,
  "y_step": 0.4,
  "z_step": 0.005,
  "grad_threshold": 0.15,
  "smoothing_window_size": 5,
  "smoothing_offset": -0.002,
  "default_cutoff_z": 3.7
}
```

Table 18 summarises the parameters that changed and the rationale tied to the trace above.

The same parameter keys are adapted by GPT-5.4 and Gemini 3 Flash with values within $\pm 5\%$ of the Opus outputs for this tunnel.

E. On-site rules baseline specification

The rules baseline uses only field-observable inputs and adjusts five parameters from the sam4tun defaults. All other parameters retain their expert-tuned values. Inputs per tunnel: nominal diameter d (design drawings), tunnel family (visual inspection), ring length L_{ring} (drawings), and segment count (counted visually). For T4–T5 tunnels, SAM template dimensions (segment_width, K_height, AB_height) are scaled by $d/5.5$ and $L_{\text{ring}}/1.2$ from the 5.5 m reference. No characteriser output, pipeline intermediate state, or domain-knowledge document is used.

Table 20

Runtime, API calls, and per-tunnel API cost. Token counts are as reported by each vendor API; USD figures use vendor list prices at submission and are indicative.

Metric	Value
LLM API calls per tunnel	5 (one per pipeline stage)
Total API calls (full ablation)	25 per condition per LLM
Wall-clock time per tunnel	~ 24 min
GPU	Single NVIDIA RTX 5060
Retraining required	None
Labelled data required	None (GT for evaluation only)

Per-tunnel tokens and cost (m+s+k, mean over 5 tunnels)

Claude Opus 4.6	64,854 in / 6,599 out, \$1.47
GPT-5.4 (high-effort reasoning)	64,414 in / 35,263 out, \$0.43
Gemini 3 Flash	83,658 in / 3,058 out, \$0.03

Table 21

Per-class IoU—Regular tunnels ($n = 3$, 6-class, Opus 4.6).

Class	sam4tun	rules	memory	m+s	m+s+k
Background	0.511	0.639	0.576	0.671	0.698
K-block	0.151	0.295	0.372	0.599	0.558
B1-block	0.217	0.256	0.338	0.643	0.613
A1-block	0.268	0.234	0.317	0.681	0.651
A2-block	0.198	0.159	0.180	0.403	0.398
A3-block	0.302	0.272	0.320	0.667	0.679
B2-block	0.229	0.412	0.317	0.710	0.723

F. Runtime, API calls, and cost

Table 20 reports the compute setup and, for one m+s+k run, the per-tunnel input/output token counts and indicative USD cost for each LLM (averaged over the 5 tunnels, five stage calls per tunnel).

Pricing assumed (per 10^6 tokens): Opus 4.x \$15 in / \$75 out; GPT-5.x \$1.25 in / \$10 out; Gemini 3-Flash \$0.30 in / \$2.50 out. The three models receive the same Markdown prompt at every stage; the spread in input-token counts reflects tokeniser density (Gemini counts more tokens on the JSON-heavy prompt), and GPT-5.4's larger output reflects internal reasoning tokens billed at the output rate with `reasoning.effort=high`.

G. Per-class IoU breakdown

Tables 21 and 22 report per-class IoU for Opus 4.6 (representative; other LLMs show the same pattern) alongside the rules baseline. For regular tunnels, rules achieve per-class IoU comparable to sam4tun while all LLM conditions improve roughly uniformly. For complex tunnels (rules $n = 2$), rules recover segment structure from near-zero for most classes but not K-block; the LLM conditions achieve further improvement on regular tunnels. B2-block remains the hardest class, requiring the 7-segment layout guidance from the knowledge component.

Table 22Per-class IoU—Complex tunnels ($n = 2$, 7-class, Opus 4.6).

Class	sam4tun	rules	memory	m+s	m+s+k
Background	0.299	0.666	0.299	0.593	0.787
K-block	0.000	0.000	0.000	0.000	0.000
B1-block	0.000	0.079	0.000	0.016	0.015
A1-block	0.000	0.039	0.000	0.047	0.026
A2-block	0.000	0.041	0.000	0.096	0.057
A3-block	0.000	0.246	0.000	0.073	0.177
A4-block	0.000	0.068	0.000	0.099	0.056
B2-block	0.000	0.059	0.000	0.063	0.041

Table 23

Performance distribution (mean across 3 LLMs).

Metric	sam4tun	rules	memory	m+s	m+s+k
Mean mIoU	0.176	0.254	0.230	0.430	0.452
Std	0.173	0.165	0.135	0.304	0.282
Min	0.037	0.119	0.068	0.099	0.156
Max	0.451	0.562	0.397	0.829	0.791

H. Performance distribution

Table 23 summarises the distribution of mIoU across tunnels (reported as the mean across the three LLMs for each condition).

I. LLM inference repeatability

Under m+s+k with temperature set to 0, we compared the primary adapted parameter JSONs (run 1) to a second LLM inference pass (run 2) on the same tunnel prompts. Run 1 parameters were left on disk before run 2 so the orchestrator could skip pipeline stages whose outputs matched run 1. Scripts: `bootstrap_repeatability_run1.py`, `run_repeatability.py`, and `reproducibility_analysis.py` -layout repeatability (repository root). Outputs are logged under `logs/{tunnel}/repeatability/`.

Table 24 summarises the first nine tunnel-model pairs recovered from held-out reruns on complex tunnels 4-4, 5-3, and 5-4 (three LLMs each). The full protocol targets 15 pairs (5 tunnels $\times$ 3 LLMs); aggregate statistics below update when the remaining pairs complete.

On these nine pairs, median critical-parameter identity was 100% (mean 88%) and mean $|\Delta \text{mIoU}|$ was 0.029, below the paired adaptation gain versus the static baseline ($\Delta \text{mIoU} \approx \text{TBD}$). Large mIoU swings on tunnel 4-4 Opus (TBD $\rightarrow$ TBD) coincide with lower parameter identity and reflect failure-recovery reruns rather than benign stochasticity; the full 15-pair batch uses a single fresh run 2 per combo.

Table 24

LLM repeatability under m+s+k (temperature 0): run 1 vs run 2 on nine held-out complex-tunnel pairs. *Crit. identity* is the fraction of the 18 critical parameters (Table 8) taking identical values in both runs.

Tunnel	LLM	mIoU run1	mIoU run2	Crit. identity
4-4	Opus-4.6	TBD	TBD	TBD
4-4	GPT-5.4	TBD	TBD	TBD
4-4	Gemini-3-Flash	TBD	TBD	TBD
5-3	Opus-4.6	TBD	TBD	TBD
5-3	GPT-5.4	TBD	TBD	TBD
5-3	Gemini-3-Flash	TBD	TBD	TBD
5-4	Opus-4.6	TBD	TBD	TBD
5-4	GPT-5.4	TBD	TBD	TBD
5-4	Gemini-3-Flash	TBD	TBD	TBD
Median critical-parameter identity				TBD
Mean $ \Delta \text{mIoU} $				TBD